Journal of Intelligent & Robotic Systems (2026) 112:95
https://doi.org/10.1007/s10846-026-02442-5

RESEARCH

Rerouting UAVs in a Dynamic Environment Based on an Optimized Interfered Fluid Dynamical System

Komsun Tamanakijprasart[1] · Sabyasachi Mondal[1] · Antonios Tsourdos[1]

Received: 25 November 2025 / Accepted: 1 August 2026
© The Author(s) 2026

Abstract
Navigating unmanned aerial vehicles (UAVs) through areas full of obstacles is a complex task. Dynamic obstacles in real-world scenarios include not only physical moving objects but also changing environmental conditions. To ensure mission success, UAVs must be able to reroute in real time to avoid collisions and high-risk zones. This need for real-time computation makes the dynamic path-planning problem even more challenging. We developed a new dynamic path-planning program based on the Interfered Fluid Dynamical System (IFDS) framework to address these issues. We improved the IFDS algorithm with both global and local path adaptation, incorporating environmental data such as weather and terrain to enhance path accuracy, and used numerical optimization methods. Every aspect of the program is expressed in a closed form, enabling efficient real-time operation even in three-dimensional environments. Our approach offers a lightweight and transparent system that allows any part of the program to be adjusted as needed, making it well-suited for real-world UAV navigation, where the system must be tailored to specific requirements or constraints.

Keywords Unmanned aerial vehicle (UAV) · 3D path planning · Dynamic path planning · Navigation · Collision avoidance · Weather avoidance

1 Introduction

Research into path planning for unmanned aerial vehicles (UAVs) has become increasingly important as they take on expanding roles in surveillance, delivery, and disaster management [1–3]. Path planning strategies have evolved from basic algorithms like A* and Dijkstra to advanced heuristic and metaheuristic methods, including genetic algorithms, particle swarm optimization, and reinforcement learning. Despite extensive research on obstacle avoidance, significant challenges remain in integrating real-time obstacle detection, adapting to dynamic environments, and maintaining computational efficiency [4–6]. Debates continue over the best balance between global path planning and local reactive avoidance, with some approaches favoring heuristic-based global strategies and others prioritizing local, learning-driven methods [7]. Addressing these gaps is essential for advancing UAV autonomy and ensuring reliable operations.

Despite the growing interest in UAVs, existing literature often overlooks the numerous challenges posed by the actual environment. When dealing with a complex 3D environment, many path-planning algorithms often struggle with long processing times, local optima, and difficulty finding collision-free paths. When a UAV encounters unpredictable obstacles, the collision avoidance algorithms deviate the UAV from its pre-planned path, which can lead to unforeseen obstacles. This raises concerns about airspace regulation and the need to devise a viable path to rejoin the original route. Figure 1 illustrates the complexities of a UAV scenario that involves navigating a 3D environment with dynamic obstacles.

To address these challenges, a real-time, executable, and optimal collision-avoidance algorithm is needed to effectively reroute the UAV to its destination while handling

Sabyasachi Mondal and Antonios Tsourdos contributed equally to this work.

✉ Komsun Tamanakijprasart
k.tamanakijprasart.263@cranfield.ac.uk

Sabyasachi Mondal
Sabyasachi.Mondal@cranfield.ac.uk

Antonios Tsourdos
Antonios.Tsourdos@cranfield.ac.uk

[1] Faculty of Engineering and Applied Science, Cranfield University, Cranfield MK43 0AL, Bedfordshire, UK

Published online: 19 September 2026

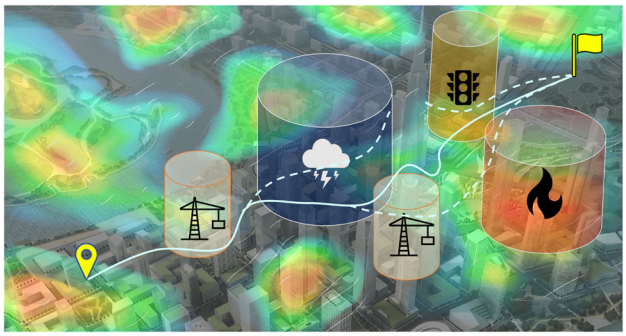


Fig. 1 Problem scenario: UAV path planning in real missions must avoid physical obstacles while adapting to dynamic conditions such as weather. However, deviations from the planned route can expose the UAV to unforeseen hazards in unfamiliar environments

dynamic obstacles and making efficient decisions across various scenarios. The objective of this study is to develop a program that executes the dynamic autorouting capability of a UAV in complex 3D environments. The problem formulation is summarized in Table 1. The program must generate a 3D path in real time, accommodating changes that may occur during the flight. It must handle static and dynamic obstacles, as well as unexpected environmental factors such as congestion and weather.

1.1 Related Work

1.1.1 Novel Path Planning Method

Motion planning for collision avoidance has been extensively researched and tested in robotics [8–11]. However, it mainly applies in 2D space and indoor environments. Tao et al. [12] propose an effective collision avoidance and trajectory recovery strategy (RCATRS) for UAVs that combines a mixed collision cone and alerting criteria. While RCATRS is real-time and effectively plans trajectories, its hard-coded logic raises questions about its adaptability in unforeseen situations. The article by Yin, C. et al. [13] explores a multi-objective path planning (MOPP) framework for UAVs that considers multiple constraints using both offline and online information. Using level set and A* methods, it generates safe, efficient paths that avoid dynamic obstacles, but has only been tested in 2D. On the other hand, the improved sparse hierarchical lazy theta* (SHLT*) algorithm [14] plans near-optimal paths in real time in unknown 3D spaces. It uses spatial modeling and can quickly adjust the path when encountering obstacles, showing potential for handling dynamic obstacles. The modified genetic algorithm (MGA) [15] enhances the exploration of the standard genetic algorithm. Despite simulating a dynamic 2D environment, the 0.70-second computation time for a 25-obstacle case raises doubts about its real-time capability in 3D environments. Shao, S. et al. [16] introduce an efficient path-planning method based on a comprehensively improved particle swarm optimization (CIPSO), claiming superior performance over MGA and other PSO variants. However, its average running time of 1.5 seconds in a 3D environment is unsuitable for real-time applications and high-speed UAVs. Jamshidi, V. et al. [17] compare and implement improved GA, improved PSO, and improved grey wolf optimization (IGWO) on the CAN bus, finding IGWO to have the highest feasibility score and the lowest execution time. Although IGWO-based UAV path planning has been simulated in various studies [18, 19], the avoidance of dynamic obstacles in complex 3D environments has not been investigated. Lastly, Hues J.S. et al. [20] propose using Generalized Vector Explicit Guidance (GENEX) [21] combined with the barrier function for real-time collision avoidance. Although static obstacles are successfully avoided in 2D and 3D scenarios, the GENEX-generated path is unsuitable for dynamic obstacles.

Recent advancements in collision-avoidance algorithms for 3D dynamic obstacles include the MADER framework [22, 23], which leverages the MINVO basis [24] and polyhedral safety boundaries to enable decentralized, asynchronous 3D trajectory planning. Zhou et al. propose a robust, real-time replanning framework for fast autonomous flight [25, 26]. Additionally, the RAPTOR framework [27] enables fast, safe flight by incorporating perception-aware replanning and risk-aware trajectory refinement. Xie et al. [28] introduce an enhanced RIME algorithm for 3D navigation in terrain, demonstrating superior collision-avoidance performance compared to competing algorithms such as PSO, GWO, and RTH. However, slow convergence remains a core issue. The Euclidean Signed Distance Field (ESDF) is a powerful method for online motion planning. The FIESTA [29]

Table 1 Problem formulation breakdown

Topic	Content
Requirements	• The UAV shall reach its destination as quickly as possible • The UAV shall avoid both static and dynamic obstacles • The UAV shall avoid high-risk areas due to environmental conditions
Constraints	• Static obstacles such as buildings • Dynamic obstacles such as other UAVs • Environment condition, such as weather and congestion • Performance of the UAV
Assumptions	• The global route is predefined and fixed • The local route will be computed onboard the UAV • Static obstacles are known from the given area map • The UAV can rapidly update its measurement of the environmental condition • No noise and uncertainty in all data

framework builds global ESDF maps incrementally while being lightweight and flexible. It offers high performance and near-optimal results using two independent queues for obstacle updates, along with Indexing Data Structures and Doubly Linked Lists within a Breadth-First Search (BFS) framework. However, it cannot be perfectly accurate due to the limitations of BFS. The EGO-Planner [30] achieves performance comparable to ESDF-based planners while being more efficient and robust, as shown by benchmarks and real-world experiments. It is an ESDF-free, gradient-based local planner that reduces computation time by directly comparing the trajectory to a collision-free guiding path rather than to a prebuilt ESDF. However, it still suffers from local minima. Zong et al. [31] introduces a multi-strategy drone planning framework for real-time obstacle avoidance in unknown environments. It uses online point cloud recognition, trajectory prediction, and a field-theory-based risk function, with a reaction time of under 3ms. The method achieves better obstacle avoidance and shorter paths, but at a higher computational cost than the EGO-planner.

While these methods are effective in addressing immediate collision avoidance with physical obstacles, they do not account for the impact of evolving environmental factors, such as weather conditions, which are often represented using intensity maps. Among novel path planning techniques, the Interfered Fluid Dynamical System (IFDS) method [32, 33] appears to offer great practicality, with low-cost computation, real-time execution, support for complex 3D environments, and dynamic obstacle avoidance. The success of the hybrid IFDS algorithm in addressing these challenges has been well documented in the literature [34–38]. The versatility of the IFDS algorithm has been demonstrated across different applications. For example, an IFDS-based route-planning approach has been proposed for an agricultural plant-protection mission involving a helicopter [39]. In this case, the IFDS algorithm was used to assess the feasibility of the generated path, accounting for the helicopter's flight performance and constraints. Another significant application is to use the IFDS as a foundation for a multi-agent formation framework [40]. Additionally, a study by Wu, J. et al. [41] refined the maneuver control for obstacle avoidance (MCOA) of fixed-wing UAVs by implementing a deep-reinforcement-learning-based reactive online decision-making mechanism built on top of the IFDS guidance law.

Current research is focused on integrating IFDS with complementary optimization algorithms. Celestini, D. et al. [37] combine the IFDS algorithm with quartic Bézier curve optimization to generate a trajectory that satisfies kinematic constraints such as maximum turn rate and climb rate for improving smoothness and safety. Yao, P. et al. use the improved GA (IGA) incorporating GWO to optimize the feasible path for autonomous underwater vehicles (AUV). Wang, Y. et al. [35] proposed using a Neural Network to adaptively adjust IFDS parameters based on environmental information. Lastly, various reinforcement learning algorithms can be applied to an adaptive interfered fluid dynamic system algorithm (AIFDS)[38].

1.1.2 Adding Environmental Constraints

The path of an unmanned aerial vehicle (UAV) can be influenced by dynamic obstacles, such as flying objects, as well as environmental factors, including rain, fog, and fire. To ensure safety, these factors must be considered in the path-planning process. This is often done by adding them as a cost function in the constrained optimization problem, which has several advantages in finding an optimal path. For example, Zhang, X. et al. [42] proposed the use of an improved fruit fly optimization (FOA) algorithm, while Xiangyin Zhang et al.[43] proposed an improved constrained differential evolution (DE) algorithm. In the most recent work by Zhang, X. et al. [44], a multi-objective particle swarm optimization algorithm with a reinforcement-learning-based multi-mode collaboration (MCMOPSO-RL) was employed. Additionally, an adaptive selection mutation-constrained differential evolution (CDE) algorithm [45], based on differential evolution, has been used to solve the problem. However, solving the constrained optimization problem alongside the path planning framework is challenging due to the iterative nature of the process, which hinders real-time computational performance. For instance, the average runtime of the algorithm in Zhang, X. et al.'s work [44] exceeded a minute, making it unsuitable for real-time applications. In the study by Wang, J. et al. [46], the dynamic environment is represented as a high-dimensional matrix, the Extended Hierarchical Graph (EHG). Real meteorological data were assigned to each layer in EHG to represent the environment at each time step, and safety possibility coefficients were assigned to individual cells. In the study by Yan et al. [47], a deep reinforcement learning approach is adopted in the UAV's reaction to the situation map, which encodes various dynamic elements in the environment.

1.2 Contribution of This Article

This study provides a unique solution for collision-free path planning in 3D space that accounts for environmental constraints, such as weather conditions. The proposed solution provides lightweight, real-time computation and complete transparency into the algorithm, allowing any part of the program to be adjusted as required, enhancing its applicability

across different scenarios. The core of the proposed solutions can be summarized as follows:

1. **Dynamic autorouting framework for UAVs**
   Firstly, an IFDS-based path-planning scheme, called dynamic autorouting, is introduced for UAVs. Its features include waypoint skipping, global and local path adaptation, position holding, and interception prediction. This scheme can rapidly reroute the UAV whenever it deviates from the planned route due to dynamic obstacles or environmental changes. The scheme is designed with a closed-form expression for guidance commands, enabling light real-time execution. This scheme provides a foundation for lightweight, collision-free path planning.
2. **Incorporating environmental constraints**
   We have derived an analytical solution for integrating generalized environmental constraints into the IFDS framework using a constraint matrix. This matrix represents the current environmental conditions and any unpredictable changes, such as rain, congestion, or no-fly zones. This representation allows for a more robust and adaptable path-planning process.
3. **Optimal IFDS path**
   The IFDS algorithm does not account for route optimality, and its hyperparameters are left as design choices. This study formulated and solved the optimization problem for the IFDS path planning. Consequently, the IFDS parameters are optimized, making it well-suited to the specific use case. Additionally, a detailed numerical simulation study was conducted to demonstrate the effectiveness of the optimal IFDS in the presence of numerous dynamic obstacles.

2 Interfered Fluid Dynamical System

In this section, the mathematical model of the Interfered Fluid Dynamical System from [48] is summarized. Let $\xi = (x, y, z) \in \mathbb{R}^3$ be the position of the UAV and $(x_0, y_0, z_0) \in \mathbb{R}^3$ be the obstacle's origin point. The trajectory is determined by iteratively updating ξ using the modified velocity field, also known as the interfered fluid velocity $\bar{\boldsymbol{u}}(\xi)$:

$$\xi_{k+1} = \xi_k + \bar{\boldsymbol{u}}(\xi)\Delta t, \qquad \bar{\boldsymbol{u}}(\xi) = \boldsymbol{M}(\xi)\boldsymbol{u}(\xi) = \boldsymbol{u}(\xi) + \boldsymbol{v}(\xi)$$

The modified fluid velocity $\bar{\boldsymbol{u}}(\xi)$ is calculated by multiplying the modulation matrix $\boldsymbol{M}(\xi)$ with the original fluid velocity $\boldsymbol{u}(\xi)$, which is equivalent to adding the original velocity $\boldsymbol{u}(\xi)$ to an additional velocity component $\boldsymbol{v}(\xi)$ caused by the obstacle. The magnitude of $\boldsymbol{v}(\xi)$ increases as the UAV approaches the obstacle. Let $d(\xi)$ represent the distance between the current UAV position ξ and the final destination $\xi_d = (x_d, y_d, z_d)$. The fluid strength, C, indicates the UAV's cruising speed. The original fluid flow is modeled as a sink located at the destination and is expressed as:

$$\boldsymbol{u}(\xi) = \left[\frac{C(x-x_d)}{d(\xi)}, \frac{C(y-y_d)}{d(\xi)}, \frac{C(z-z_d)}{d(\xi)}\right]^{\mathsf{T}}, \quad d(\xi) = \sqrt{(x-x_d)^2 + (y-y_d)^2 + (z-z_d)^2} \tag{1}$$

The definition of the modulation matrix $\boldsymbol{M}(\xi)$ is given by:

$$\boldsymbol{M}(\xi) = \boldsymbol{I} - \frac{\boldsymbol{n}(\xi)\boldsymbol{n}(\xi)^{\mathsf{T}}}{|\Gamma(\xi)|^{\frac{1}{\rho(\xi)}}\boldsymbol{n}(\xi)^{\mathsf{T}}\boldsymbol{n}(\xi)} + \frac{\boldsymbol{t}(\xi)\boldsymbol{n}(\xi)^{\mathsf{T}}}{|\Gamma(\xi)|^{\frac{1}{\sigma(\xi)}}\|\boldsymbol{t}(\xi)\|\|\boldsymbol{n}(\xi)\|} \tag{2}$$

where $\boldsymbol{n}(\xi)$, $\boldsymbol{t}(\xi)$, $\rho(\xi)$, and $\sigma(\xi)$ are defined as follows:

$$\boldsymbol{n}(\xi) = \left[\frac{\partial\Gamma(\xi)}{\partial x}, \frac{\partial\Gamma(\xi)}{\partial y}, \frac{\partial\Gamma(\xi)}{\partial z}\right]^{\mathsf{T}}, \quad \boldsymbol{t}(\xi) = \left[\frac{\partial\Gamma(\xi)}{\partial y}, -\frac{\partial\Gamma(\xi)}{\partial x}, 0\right]^{\mathsf{T}}$$
$$\rho(\xi) = \rho_0 \exp\left(1 - \frac{1}{d_0(\xi)d(\xi)}\right), \quad \sigma(\xi) = \sigma_0 \exp\left(1 - \frac{1}{d_0(\xi)d(\xi)}\right)$$

ρ_0 is the repulsive parameter, σ_0 is the tangential parameter, and $d_0(\xi)$ is the distance between the obstacle's boundary and the current position of the UAV ξ. Additionally, $\boldsymbol{n}(\xi)$ and $\boldsymbol{t}(\xi)$ represent the unit normal vector and the unit tangential vector outward on the surface of the obstacle, respectively. ρ_0 defines the range within which the path responds to the presence of an obstacle. A larger ρ_0 results in a greater deviation from the original flow. Conversely, σ_0 governs the degree of flexibility for lateral path deviations. A higher value of

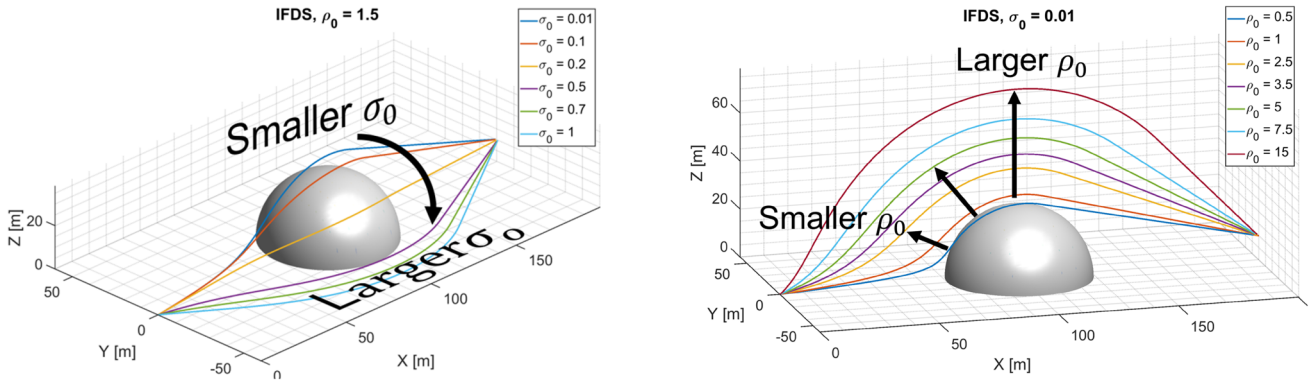


Fig. 2 3D paths with varied IFDS parameters

σ_0 makes the path more adaptable to sideways deviations. The impact of the tuning parameters ρ_0 and σ_0 is visually demonstrated in Fig. 2.

In some cases, following the shape of an obstacle is not desired. Yao, P. et al.[33] proposed the shape-following parameter SF $\in \{0, 1\}$ so that SF $= 1$ when the shape-following is demanded, and SF $= 0$ when it is not. The effect of this feature is shown in Fig. 3. The modulation matrix from equation (2) is then redefined as:

$$\boldsymbol{M}(\xi) = \begin{cases} \boldsymbol{I} - \dfrac{\boldsymbol{n}(\xi)\boldsymbol{n}(\xi)^{\mathsf{T}}}{|\Gamma(\xi)|^{\frac{1}{\rho(\xi)}}\boldsymbol{n}(\xi)^{\mathsf{T}}\boldsymbol{n}(\xi)} + \dfrac{\boldsymbol{t}(\xi)\boldsymbol{n}(\xi)^{\mathsf{T}}}{|\Gamma(\xi)|^{\frac{1}{\sigma(\xi)}}\|\boldsymbol{t}(\xi)\|\|\boldsymbol{n}(\xi)\|}, & \text{if } \boldsymbol{n}(\xi)^{\mathsf{T}}\boldsymbol{v}(\xi) < 0 \text{ or SF } = 1 \\ \boldsymbol{I}, & \text{if } \boldsymbol{n}(\xi)^{\mathsf{T}}\boldsymbol{v}(\xi) \geq 0 \text{ and SF } = 0 \end{cases}$$

For K obstacles in the environment, let the object modeling function of the kth obstacle be $\Gamma_k(\xi) = 1, k = 1, 2, ..., K$, and the interfered fluid flow for the kth obstacle is

$$\bar{\boldsymbol{u}}_k(\xi) = \boldsymbol{M}_k(\xi)\boldsymbol{u}(\xi), \quad \boldsymbol{M}_k(\xi) = \boldsymbol{I} - \frac{\boldsymbol{n}_k(\xi)\boldsymbol{n}_k(\xi)^{\mathsf{T}}}{|\Gamma_k(\xi)|^{\frac{1}{\rho_k(\xi)}}\boldsymbol{n}_k(\xi)^{\mathsf{T}}\boldsymbol{n}_k(\xi)} + \frac{\boldsymbol{t}_k(\xi)\boldsymbol{n}_k(\xi)^{\mathsf{T}}}{|\Gamma_k(\xi)|^{\frac{1}{\sigma_k(\xi)}}\|\boldsymbol{t}_k(\xi)\|\|\boldsymbol{n}_k(\xi)\|}$$

Let $\omega_k(\xi)$ be the weighting coefficient of the kth obstacle, and $\bar{\omega}_k(\xi)$ be the normalized relative weighting coefficient:

$$\omega_k(\xi) = \prod_{i=1, i\neq k}^{K} \frac{\Gamma_i(\xi) - 1}{(\Gamma_k(\xi) - 1) + (\Gamma_i(\xi) - 1)}, \quad \tilde{\omega}_k(\xi) = \frac{\omega_k(\xi)}{\sum_{i=1}^{K}\omega_i(\xi)}$$

The weighted average of the velocity field ensures that all obstacles in the area are avoided. The IFDS path for avoiding multiple obstacles can be seen in Fig. 4.

$$\boldsymbol{M}(\xi) = \sum_{k=1}^{K} \tilde{\omega}_k(\xi)\boldsymbol{M}_k(\xi)$$

3 Modeling of Environment

3.1 Modeling of Static Environment

Many objects, such as mountains, trees, buildings, and aircraft, can be simplified into cylinders, cones, spheres, parallelepipeds, or combinations of these shapes. By doing so, the obstacle-avoidance problem for the IFDS method can be formulated by solving for fluid streamlines rather than using simplified shapes. The object modeling function $\Gamma : \mathbb{R}^3 \rightarrow \mathbb{R}$ can be expressed as follows:

$$\Gamma(\xi) = \left(\frac{x - x_0}{a}\right)^{2p} + \left(\frac{y - y_0}{b}\right)^{2q} + \left(\frac{z - z_0}{c}\right)^{2r} = 1 \quad (3)$$

where $\xi = (x, y, z)$ is the UAV position, $\xi_0 = (x_0, y_0, z_0)$ is object's origin point, (a, b, c) are object's axis length, and (p, q, r) are index parameters which give various shapes as follow:

$$\text{object} = \begin{cases} \text{sphere}, & \text{if } p = q = r = 1. \\ \text{cylinder}, & \text{if } p = q = 1 \text{ and} r > 1. \\ \text{cone}, & \text{if } p = q = 1 \text{ and } r < 1. \\ \text{parallelepiped}, & \text{if } p > 1, q > 1 \text{ and } r > 1. \end{cases} \quad (4)$$

$\Gamma(\xi)$ can be viewed as a function to detect the boundary of the obstacle where $\Gamma(\xi) = 1$ at the object's surface and $\Gamma(\xi) > 1$ when away from the object.

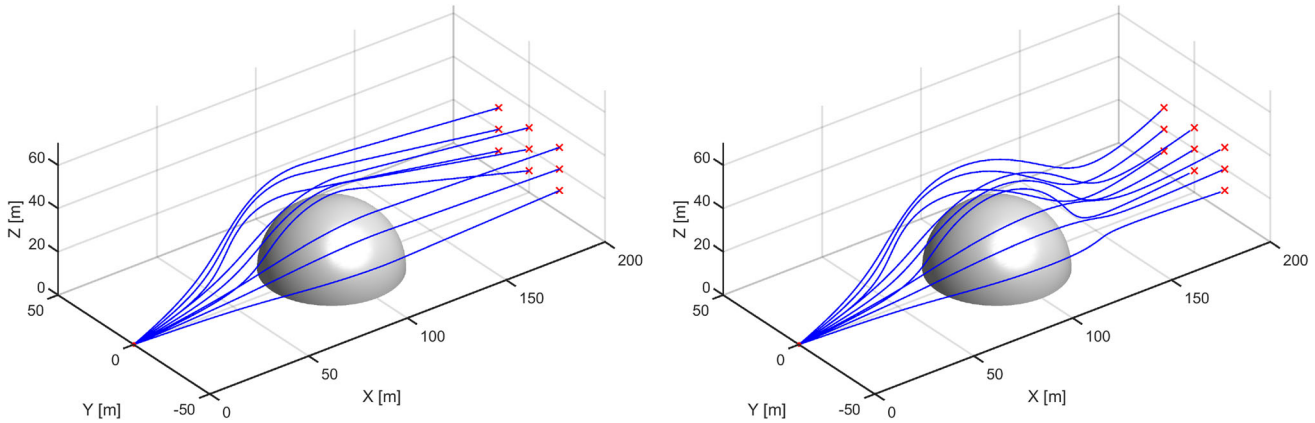


Fig. 3 The IFDS path with and without the shape-following feature. Left: SF = 0. Right: SF = 1

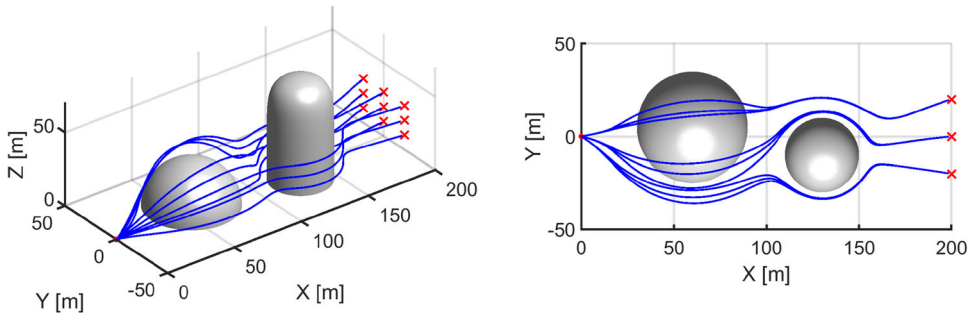


Fig. 4 IFDS paths for multiple obstacles avoidance

3.2 Modeling of Dynamically Changing Environment

This work introduces a matrix representing environmental constraints with controlled random patterns over time. The proposed approach employs the 2D Fast Fourier Transform (FFT) and inverse Fast Fourier Transform (iFFT) to generate spatially coherent images. The randomness control parameter is manipulated by selectively zeroing out specific frequency components ($nx1, nx2, ny1, ny2$) in the frequency domain, resulting in dynamic, evolving patterns. The process is detailed in algorithm 1. The resulting cost maps are scaled to fit the desired intensity range, yielding generalized representations of environmental constraints, such as weather and GNSS signal degradation.

The randomness control parameter f has been introduced to adjust the randomness of the generated weather maps. A lower value of f results in a more random pattern, while a higher value leads to smoother patterns, as demonstrated in Fig. 5. Furthermore, introducing sinusoidal oscillations in the zeroed-out regions allows for dynamic and evolving cost patterns over time, as shown in Fig. 6. Following the frequency domain manipulation, the modified cost map is transformed back to the spatial domain using inverse 2D FFT ($P \leftarrow \mathbf{iFFT2}(P)$). This step restores the cost map to its original spatial representation, incorporating the introduced pattern modifications. Finally, the min-max normalization technique is employed to scale the values to the range 0 to 1.

3.2.1 Constraint Matrix Derivation

Let the levels of environmental obstruction in an area of size $N_x \times N_y\ m^2$ be represented by a matrix $\mathbf{A} \in \mathbb{R}^{N_y \times N_x}$, containing normalized values ranging from 0 to 1. The function $\Omega : \mathbb{R}^3 \rightarrow \mathbb{R}$ is then introduced to interpolate the data at arbitrary location $\xi = (x,\ y,\ z)$ within the space using bicubic interpolation:

$$\Omega(\xi) = \sum_{i=1}^{N_x} \sum_{j=1}^{N_y} a_{ij}\ x^i y^j \tag{5}$$

where the coefficients $a_{ij} \in \mathbb{R}$ are determined from 16 conditions based on surrounding data points. Two crucial parameters are also introduced: the upper boundary value, denoted as B_U, and the lower boundary value, denoted as B_L. These parameters are pivotal in governing the extent of the constraint's impact. Specifically, values below B_L are deemed negligible, while those exceeding B_U are considered the most critical threat. The impact of setting these lower and upper boundary values is illustrated in Fig. 7.

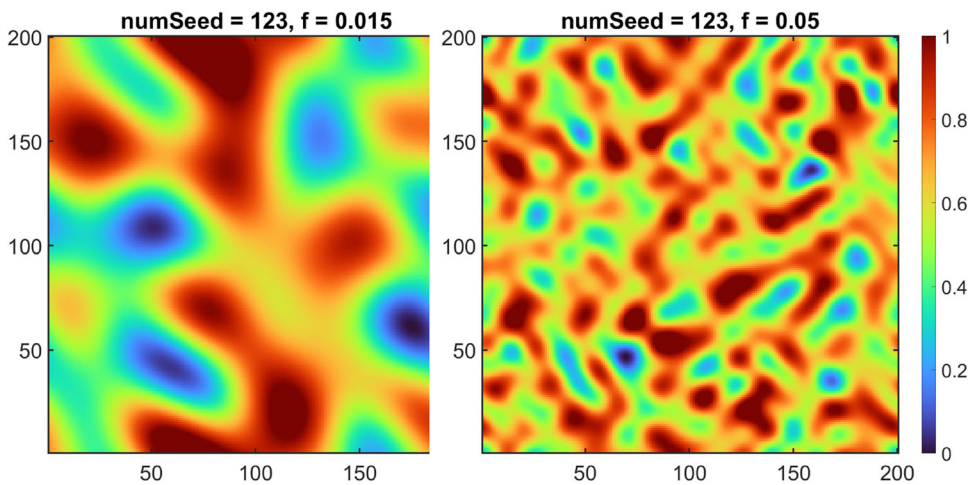


Fig. 5 Generated constraint matrices. Left: $f = 0.015$. Right: $f = 0.05$

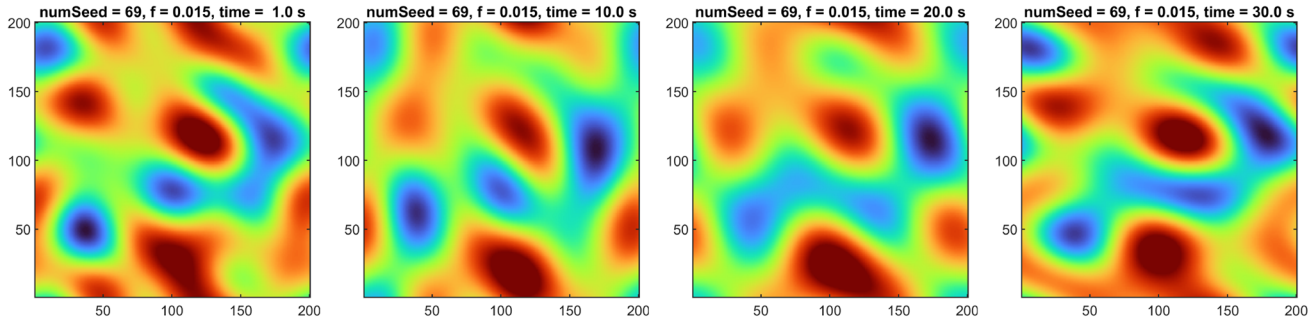


Fig. 6 Evolution of constraint matrices over time

However, setting B_U too low may cause block formation in the area, rendering path planning infeasible. A concrete illustration of this concern is shown in Fig. 8, where introducing a new boundary obstructs the entire area, rendering the path-planning task infeasible. To ensure the viability of the path-planning process, professionals must inspect the area matrix after setting the boundary values. Incorporating the environmental constraint $\Omega(\xi)$ into consideration, the modified boundary function $\Gamma_p : \mathbb{R}^3 \rightarrow \mathbb{R}$ is proposed having the form of:

$$\Gamma_p(\xi) = \Gamma(\xi) - g(\Omega(\xi)) \leq \Gamma(\xi) \tag{6}$$

with distinct outcomes as follows:

1. For $B_L < \Omega(\xi) < B_U$, the value of $\Gamma_p(\xi)$ is confined to the interval $(1, \Gamma(\xi))$
2. In the case of $\Omega(\xi) \leq B_L$, the boundary function remains unaltered, retaining its original form, expressed as $\Gamma_p(\xi) = \Gamma(\xi)$
3. When $\Omega(\xi) \geq B_U$, $\Gamma_p(\xi) = 1$, thereby creating a new boundary in the field

The function $g : \mathbb{R} \rightarrow \mathbb{R}$ is a constraint-shaping function that controls the output of constraint values. It influences the path response to avoid environmental constraints. The shifted exponential function is adopted, providing a powerful means to model a response that remains unresponsive under certain conditions and then adapts to dynamic environments once the constraints are lifted. The shifted exponential function is expressed as:

$$f(x) = k(e^{n(x-c)} - 1) \tag{7}$$

In this equation, n is a positive constant determining the exponential growth rate. k is a constant multiplier to control the amplitude of the exponential function. The influence of assigning various values of n and k is demonstrated in Fig. 9. Finally, c denotes the value of x at which the graph starts to increase exponentially. Therefore, the constraint-shaping function $g(\Omega(\xi))$ assumes a general form:

$$g(\Omega(\xi)) = k\left(e^{n(\Omega(\xi)-B_L)} - 1\right) \tag{8}$$

By subtracting B_L from the exponential function, $g(\Omega(\xi))$ effectively reaches a value of zero when $\Omega(\xi) = B_L$, and for $\Omega(\xi) > B_L$, the function exhibits exponential growth, allowing the path planning algorithm to respond adaptively to the dynamic environment. From the desired behavior, two boundary conditions are created:

$$g(\Omega(\xi) = B_U) = \Gamma(\xi) - 1 \tag{BC1}$$

$$g(\Omega(\xi) = B_L) = 0 \tag{BC2}$$

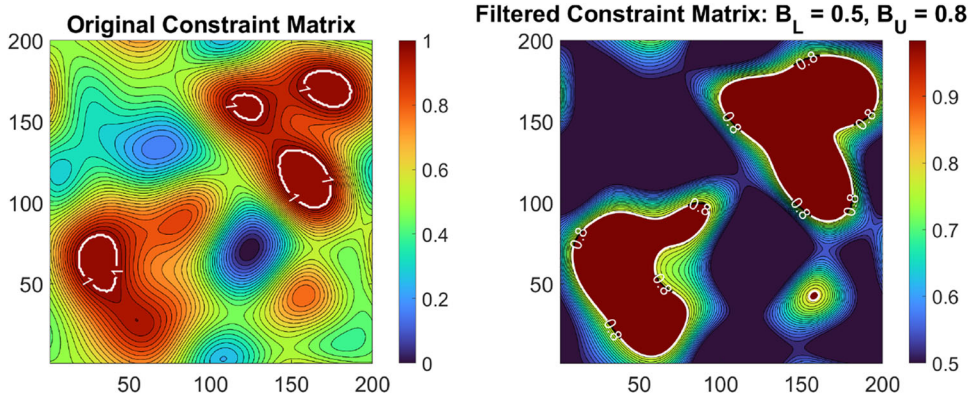


Fig. 7 The constraint matrix after setting the upper and lower boundary values

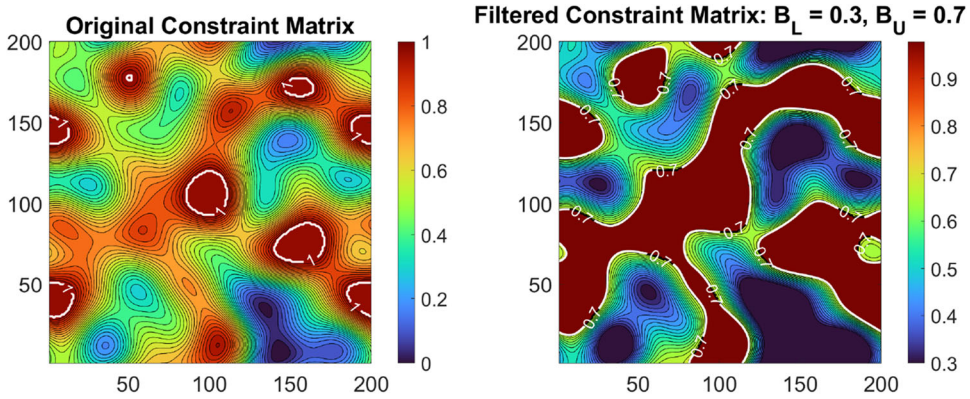


Fig. 8 The effect of setting a low value for B_U on path-planning feasibility

These boundary conditions establish a framework for superposing the object's boundary function and the environmental constraint boundary function, thereby harmonizing their effects within the algorithm. The solution form of $g(\Omega(\xi))$ can be obtained by solving for (8) satisfying two boundary conditions (BC1) and (BC2). First, let's consider (BC2).

$$g(\Omega(\xi) = B_L) = k\left(e^0 - 1\right) = 0$$

It can be seen that (BC2) is already satisfied. Next, from (BC1) we obtain

$$\begin{aligned} k\left(e^{n(B_U - B_L)} - 1\right) &= \Gamma(\xi) - 1 \\ e^{n(B_U - B_L)} &= \frac{\Gamma(\xi) - 1}{k} + 1 \\ n &= \frac{1}{B_U - B_L} \ln\left(\frac{\Gamma(\xi) - 1}{k} + 1\right) \end{aligned} \tag{9}$$

By substituting (9) into (8), $g(\Omega(\xi))$ can be expressed as:

$$g(\Omega(\xi)) = k\left\{\exp\left(\frac{\Omega(\xi) - B_L}{B_U - B_L} \ln\left(\frac{\Gamma(\xi) - 1}{k} + 1\right)\right) - 1\right\} \tag{10}$$

Finally, the modified boundary function $\Gamma_p : \mathbb{R}^3 \to \mathbb{R}$ is found to be as the following form:

$$\Gamma_p(\xi) = \Gamma(\xi) - k\left\{\exp\left(\frac{\Omega(\xi) - B_L}{B_U - B_L} \ln\left(\frac{\Gamma(\xi) - 1}{k} + 1\right)\right) - 1\right\} \tag{11}$$

The normal and tangential vectors must be recalculated. According to the modified boundary function $\Gamma_p(\xi)$, the gradient terms can be expressed as follows:

$$\begin{aligned} \frac{\partial \Gamma_p(\xi)}{\partial x} = \frac{\partial \Gamma(\xi)}{\partial x} + k \exp\left\{\frac{B_L - \Omega(\xi)}{B_L - B_U} ln\left(\frac{\Gamma(\xi) - 1}{k} + 1\right)\right\} \\ \left\{\frac{\ln\left(\frac{\Gamma(\xi)-1}{k} + 1\right)}{B_L - B_U} \frac{\partial \Omega(\xi)}{\partial x}\right. \\ \left. - \frac{B_L - \Omega(\xi)}{(\Gamma(\xi) - 1 + k)(B_L - B_U)} \frac{\partial \Gamma(\xi)}{\partial x}\right\} \end{aligned}$$

$$\begin{aligned} \frac{\partial \Gamma_p(\xi)}{\partial y} = \frac{\partial \Gamma(\xi)}{\partial y} + k \exp\left\{\frac{B_L - \Omega(\xi)}{B_L - B_U} \ln\left(\frac{\Gamma(\xi) - 1}{k} + 1\right)\right\} \\ \left\{\frac{\ln\left(\frac{\Gamma(\xi)-1}{k} + 1\right)}{B_L - B_U} \frac{\partial \Omega(\xi)}{\partial y}\right. \\ \left. - \frac{B_L - \Omega(\xi)}{(\Gamma(\xi) - 1 + k)(B_L - B_U)} \frac{\partial \Gamma(\xi)}{\partial y}\right\} \end{aligned}$$

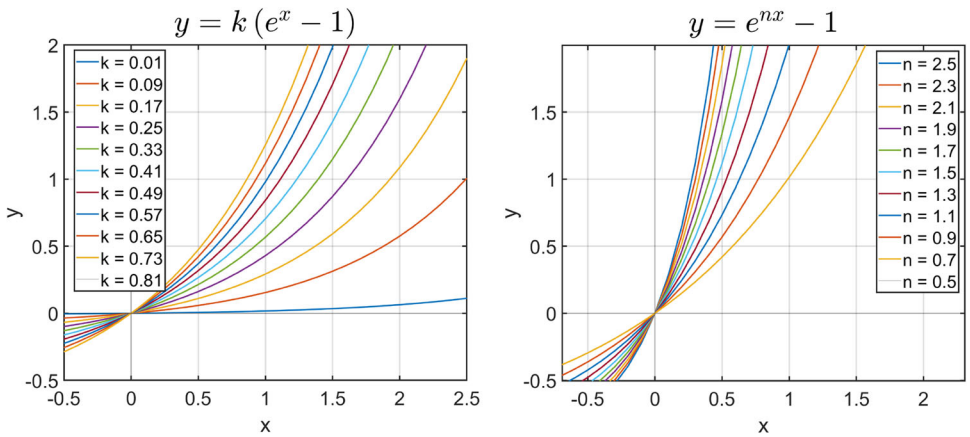


Fig. 9 The exponential graphs with various constant values

$$\frac{\partial \Gamma_p(\xi)}{\partial z} = \frac{\partial \Gamma(\xi)}{\partial z} + k \exp\left\{\frac{B_L - \Omega(\xi)}{B_L - B_U} \ln\left(\frac{\Gamma(\xi) - 1}{k} + 1\right)\right\} \left\{-\frac{B_L - \Omega(\xi)}{(\Gamma(\xi) - 1 + k)(B_L - B_U)} \frac{\partial \Gamma(\xi)}{\partial z}\right\}$$

The gradients of $\Omega(\xi)$ along the x and y directions can be approximated numerically. The impact of this superpositioning of the object boundary function and the environmental constraint boundary function is visually demonstrated in Fig. 10. The original distributions of the object and the constraint matrix are depicted in the top-middle and top-right subfigures, respectively.

The superimposed distribution of the new boundary function Γ_p is shown in the subfigures in the bottom row. The illustration clearly shows the emergence of a new boundary function that successfully integrates the object's and the environment's boundaries in the XY plane. Remarkably, this integration is achieved while preserving the distribution in the YZ and XZ planes, thereby offering a comprehensive representation of the object's influence within the environment.

4 Consistency and Validity of the Modified Boundary Function

This section presents mathematical proofs demonstrating the conservation property of the original Interfered Fluid Dynamical Systems (IFDS). For brevity, whenever the spatial dependence is clear, we will write Γ, Γ_p, Ω in place of $\Gamma(\boldsymbol{\xi})$, $\Gamma_p(\boldsymbol{\xi})$, $\Omega(\boldsymbol{\xi})$. We further assume $\Gamma > 1$ in the interior of the working domain, and the surface $\Gamma = 1$ is handled separately as a trivial case. A useful auxiliary quantity is

$$L := \ln\left(\frac{\Gamma - 1}{k} + 1\right), \qquad \lambda := \frac{\Omega - B_L}{B_U - B_L}, \tag{12}$$

so that Eq. 10 becomes the compact expression

$$g = k\left(e^{\lambda L} - 1\right). \tag{13}$$

Since $\Gamma > 1$ and $k > 0$, we have $L > 0$; and for $\Omega \in [B_L, B_U]$, we have $\lambda \in [0, 1]$.

4.1 Piecewise Behavior

Theorem 1 *(Three-case behavior) Let* $\Gamma(\boldsymbol{\xi}) > 1$, $k > 0$, $B_L < B_U$. *With* Γ_p *defined by Eq. 11,*

$$\Gamma_p(\boldsymbol{\xi}) = \begin{cases} \Gamma(\boldsymbol{\xi}), & \Omega(\boldsymbol{\xi}) \le B_L, \\ \in \big(1,\ \Gamma(\boldsymbol{\xi})\big), & B_L < \Omega(\boldsymbol{\xi}) < B_U, \\ 1, & \Omega(\boldsymbol{\xi}) \ge B_U. \end{cases}$$

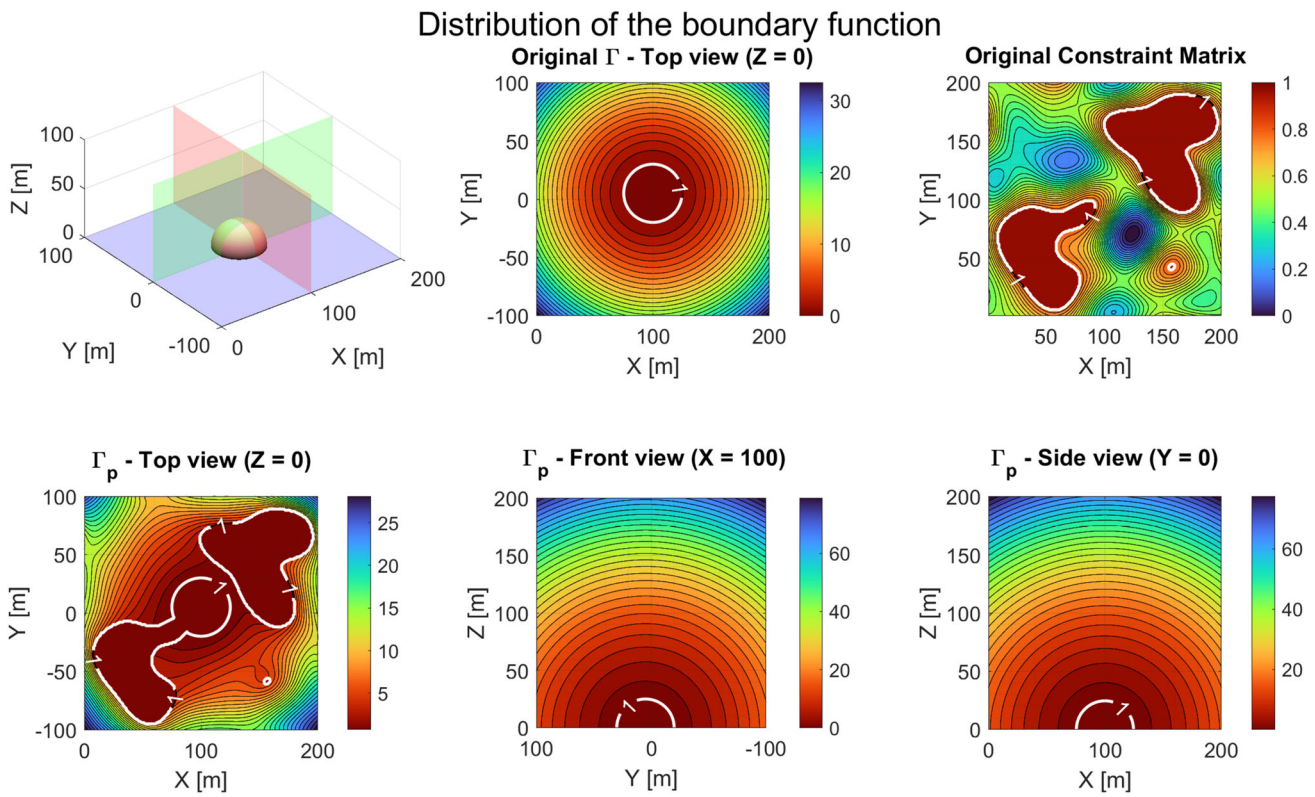


Fig. 10 Superposition of object and environmental constraint boundaries

In particular, the two boundary conditions

$$g\big(\Omega = B_L\big) = 0, \tag{BC2}$$

$$g\big(\Omega = B_U\big) = \Gamma(\boldsymbol{\xi}) - 1 \tag{BC1}$$

are both satisfied by the closed form Eq. 10.

Proof Using the compact form Eq. 13 together with the definitions Eq. 12:
Case $\Omega = B_L$**.** Then $\lambda = 0$, giving

$$g \;=\; k\Big(e^{0\cdot L} - 1\Big) \;=\; 0,$$

so $\Gamma_p = \Gamma - 0 = \Gamma$. This simultaneously proves (BC2).
Case $\Omega = B_U$**.** Then $\lambda = 1$, giving

$$g \;=\; k\Big(e^{L} - 1\Big) \;=\; k\left(\frac{\Gamma - 1}{k} + 1 - 1\right) \;=\; \Gamma - 1,$$

so $\Gamma_p = \Gamma - (\Gamma - 1) = 1$. This simultaneously proves (BC1).
Case $B_L < \Omega < B_U$**.** Then $\lambda \in (0, 1)$ and, since $L > 0$, $\lambda L \in (0, L)$. The exponential is strictly increasing, hence

$$e^{\lambda L} \;\in\; (e^0,\, e^L) \;=\; \big(1,\; \tfrac{\Gamma-1}{k} + 1\big)\,.$$

Multiplying by k and subtracting k,

$$g \;=\; k(e^{\lambda L} - 1) \;\in\; \big(0,\; \Gamma - 1\big),$$

so $\Gamma_p = \Gamma - g \in (1, \Gamma)$.
Clipping cases $\Omega \le B_L$ **and** $\Omega \ge B_U$**.** These are handled by definition (values below B_L are ignored, values above B_U are capped), consistent with the enumerated behavior above.

Collecting the three cases proves the theorem. □

Corollary 2 *(Continuity at the thresholds)* $\Gamma_p(\boldsymbol{\xi})$ *is continuous in* Ω *at* $\Omega = B_L$ *and* $\Omega = B_U$*: the left- and right-hand limits agree with the clipped values.*

Proof From Theorem 1, $\lim_{\Omega \to B_L^+} \Gamma_p = \Gamma = \Gamma_p|_{\Omega \le B_L}$ and $\lim_{\Omega \to B_U^-} \Gamma_p = 1 = \Gamma_p|_{\Omega \ge B_U}$. □

4.2 Preservation of Normal–Tangent Orthogonality

The IFDS streamline-deflection mechanism relies on the normal $\mathbf{n}$ and the tangential direction $\mathbf{t}$ being mutually orthogonal. The next proposition shows that this property is preserved when the original Γ is replaced by the modified Γ_p, regardless of the specific shapes of Γ and Ω.

Proposition 3 *(Algebraic orthogonality of* $\mathbf{n}_p$ *and* $\mathbf{t}_p$*) Define the modified normal and tangent vectors*

$$\mathbf{n}_p = \left[\frac{\partial \Gamma_p}{\partial x},\, \frac{\partial \Gamma_p}{\partial y},\, \frac{\partial \Gamma_p}{\partial z}\right]^{\mathsf{T}}, \qquad \mathbf{t}_p \;=\; \left[\frac{\partial \Gamma_p}{\partial y},\, -\frac{\partial \Gamma_p}{\partial x},\, 0\right]^{\mathsf{T}}$$

Then

$$\mathbf{n}_p^{\top}\mathbf{t}_p \;=\; 0$$

identically, i.e. for every *choice of* Γ, Ω, B_L, B_U, *and* k *satisfying the standing hypotheses.*

Proof Write $a = \partial\Gamma_p/\partial x, b = \partial\Gamma_p/\partial y, c = \partial\Gamma_p/\partial z$. Then

$$\mathbf{n}_p^{\top}\mathbf{t}_p \;=\; a \cdot b \;+\; b \cdot (-a) \;+\; c \cdot 0 \;=\; ab - ba \;=\; 0.$$

The identity is purely algebraic and does not depend on the specific form of Γ_p. □

Remark Proposition 3 shows that introducing the environmental correction $g(\Omega)$ does not violate the geometric assumptions underpinning the IFDS streamline construction. The modified field Γ_p can therefore be used as a drop-in replacement for Γ in all downstream equations that use $\mathbf{n}$ and $\mathbf{t}$.

4.3 Validity of Γ_p as an IFDS Boundary Function

The IFDS formulation requires a boundary function Γ satisfying $\Gamma \ge 1$, with equality exactly on the obstacle surface. We show that this property is preserved by Γ_p.

Proposition 4 *(Lower bound on* Γ_p*) If* $\Gamma(\boldsymbol{\xi}) \ge 1$, $B_L < B_U$, *and* $k > 0$, *then*

$$\Gamma_p(\boldsymbol{\xi}) \;\ge\; 1 \qquad \textit{for every } \boldsymbol{\xi} \in \mathbb{R}^3,$$

with equality if and only if either $\Gamma(\boldsymbol{\xi}) = 1$ *(original surface) or* $\Omega(\boldsymbol{\xi}) \ge B_U$ *(environmental boundary).*

Proof We consider the three regimes from Theorem 1.
Regime $\Omega \le B_L$. Here $\Gamma_p = \Gamma \ge 1$ by assumption, with equality if and only if $\Gamma = 1$.
Regime $\Omega \ge B_U$. Here $\Gamma_p = 1$ by definition.
Regime $B_L < \Omega < B_U$. By Theorem 1, $\Gamma_p \in (1, \Gamma) \subset (1, \infty)$, so $\Gamma_p > 1$ strictly.
In all three cases $\Gamma_p \ge 1$, and the equality cases are exactly as stated. □

Remark Proposition 4 ensures that the zero-level set of $\Gamma_p - 1$ defines a well-posed obstacle surface (the union of the original object surface and the newly formed environmental boundary), so that the IFDS potential-flow machinery remains applicable without modification.

4.4 Preservation of the IFDS Path Convergence

We adopt the standard IFDS attractive velocity field

$$\boldsymbol{u}(\xi) \;=\; -C\,\frac{\xi-\xi_d}{\|\xi-\xi_d\|}, \qquad C>0, \tag{14}$$

which always points from the current position $\xi \in \mathbb{R}^3$ to the destination ξ_d. The modified velocity field is

$$\bar{\boldsymbol{u}}_p(\xi) \;=\; \boldsymbol{M}_p(\xi)\,\boldsymbol{u}(\xi),$$

where $\boldsymbol{M}_p(\xi) \in \mathbb{R}^{3\times 3}$ is the IFDS modulation matrix constructed from $\nabla\Gamma_p$ and the associated tangent vector.

Theorem 5 *(Preservation of convergence) Suppose that on the working domain $\mathcal{D} = \mathbb{R}^3 \setminus \{\xi_d\}$ the modulated velocity satisfies the angular admissibility condition*

$$\boldsymbol{u}(\xi)^{\mathsf{T}}\,\bar{\boldsymbol{u}}_p(\xi) \;\geq\; 0 \qquad \forall\,\xi \in \mathcal{D}, \tag{15}$$

with equality only on a measure-zero stagnation set $\mathcal{S}$. Then every trajectory $\xi(t)$ of $\dot{\xi} = \bar{\boldsymbol{u}}_p(\xi)$ starting in $\mathcal{D}$ remains in $\mathcal{D}$, the distance $\|\xi(t)-\xi_d\|$ is non-increasing, and generically $\xi(t) \to \xi_d$ as $t \to \infty$.

Proof Consider the Lyapunov candidate $V(\xi) = \frac{1}{2}\|\xi-\xi_d\|^2$, which is positive definite about ξ_d and radially unbounded.
Step 1 (Non-increase). Along trajectories,

$$\dot{V} = (\xi-\xi_d)^{\mathsf{T}}\bar{\boldsymbol{u}}_p(\xi).$$

Substituting $\xi-\xi_d = -\|\xi-\xi_d\|\,\boldsymbol{u}(\xi)/C$ from Eq. 14 gives

$$\dot{V} = -\frac{\|\xi-\xi_d\|}{C}\,\boldsymbol{u}(\xi)^{\mathsf{T}}\bar{\boldsymbol{u}}_p(\xi) \;\leq\; 0, \tag{16}$$

since the prefactor $\|\xi-\xi_d\|/C$ is positive on $\mathcal{D}$ and the inequality follows from (15). Therefore, the squared-distance to the destination is non-increasing along every trajectory. Consequently, the sub-level set $V \leq V(\xi(0)) \subset \mathcal{D} \cup \{\xi_d\}$ is forward-invariant, and the trajectory remains in $\mathcal{D}$ for all $t \geq 0$.
Step 2 (Limit set). Since $\dot{V} \leq 0$ and $V \geq 0$, LaSalle's invariance principle [49] implies that every trajectory converges to the largest invariant set $\mathcal{E} \subseteq \mathcal{D} \cup \{\xi_d\}$ contained in $\{\xi : \dot{V} = 0\}$. From Eq. 16,

$$\dot{V} = 0 \Longleftrightarrow \boldsymbol{u}(\xi)^{\mathsf{T}}\bar{\boldsymbol{u}}_p(\xi) = 0 \Longleftrightarrow \xi = \xi_d \text{ or } \xi \in \mathcal{S},$$

where $\mathcal{S} \subset \mathcal{D}$ is the stagnation set on which the modulated velocity becomes orthogonal to the attractive velocity. Hence $\mathcal{E} = \{\xi_d\} \cup \mathcal{S}_{\text{inv}}$, with $\mathcal{S}_{\text{inv}}$ the invariant part of $\mathcal{S}$.
Step 3 (Genericity). Points of $\mathcal{S} \setminus \{\xi_d\}$ are saddle-type equilibria of the modulated flow whose center–stable manifolds have positive codimension [48]. The set of initial conditions attracted to $\mathcal{S}_{\text{inv}}$ is therefore contained in a finite union of lower-dimensional manifolds and has measure zero. For almost every $\xi(0) \in \mathcal{D}$, the trajectory satisfies $\xi(t) \to \xi_d$.

Steps 1–3 together show that the modified flow $\bar{\boldsymbol{u}}_p$ preserves the convergence guarantees of the unmodified IFDS algorithm. □

Remark Condition (15) requires that the modulated velocity never points away from the goal. For the modulation matrix constructed from $\nabla\Gamma_p$ in §3.2.1, this property is inherited from the original IFDS formulation: the matrix has spectral form $\boldsymbol{M}_p = \boldsymbol{I} - \rho\,\boldsymbol{n}_p\boldsymbol{n}_p^{\mathsf{T}}/\|\boldsymbol{n}_p\|^2 + \sigma\,\boldsymbol{t}_p\boldsymbol{n}_p^{\mathsf{T}}/(\|\boldsymbol{t}_p\|\|\boldsymbol{n}_p\|)$, whose eigenvalue along $\boldsymbol{u}$ remains non-negative for the standard repulsive and tangential gains ρ, σ [48]. Proposition 3 confirms that the geometric structure required for this spectral argument is preserved under the environmental modification.

Remark (Stagnation caveat) The genericity statement in Step 3 mirrors a limitation shared by all potential-flow guidance methods: saddle-type stagnation points behind obstacles cannot be excluded by Lyapunov analysis alone. In practice, time-discretization and the IFDS replanning loop perturb trajectories off the stable manifolds of these saddles, making persistent stagnation a measure-zero event in simulation and flight.

4.5 Monotonicity of *g* and the Adaptive Response

Proposition 6 *(Strict monotonicity) For $\Omega \in [B_L, B_U]$, the constraint-shaping function g defined in Eq. 10 is strictly increasing in Ω; equivalently, the modified boundary function $\Gamma_p = \Gamma - g$ is strictly decreasing in Ω.*

Proof Using (13), differentiate with respect to Ω:

$$\frac{dg}{d\Omega} \;=\; k\cdot\frac{L}{B_U-B_L}\,\exp\!\left(\frac{\Omega-B_L}{B_U-B_L}\,L\right).$$

Under the standing hypotheses $k>0$, $B_U > B_L$, and $\Gamma > 1$ (so that $L>0$), every factor is strictly positive, giving $dg/d\Omega > 0$ on $[B_L, B_U]$. Hence g is strictly increasing, and $\Gamma_p = \Gamma - g$ is strictly decreasing. □

Remark Proposition 6 formalizes the intuitive claim that g exhibits a smooth, monotone "ramp-up" behavior: negligible influence near $\Omega = B_L$, then adaptive exponential growth until the cap at $\Omega = B_U$. This monotonicity underlies the path-planner's ability to respond progressively to increasing environmental constraints rather than switching discontinuously.

5 Optimization of the IFDS Path

The path produced by the IFDS algorithm is not, in general, the shortest feasible path. This section formulates an optimization problem over the IFDS tuning parameters (ρ_0, σ_0) to minimize path length.

5.1 Objective Function

The discrete path length is

$$L = \sum_{k=1}^{N} \|\xi_{k+1} - \xi_k\|. \tag{17}$$

Since the IFDS propagation rule

$$\xi_{k+1} = \xi_k + \bar{\boldsymbol{u}}(\xi)\,\Delta t, \qquad \bar{\boldsymbol{u}}(\xi) = \boldsymbol{M}(\xi)\,\boldsymbol{u}(\xi), \tag{18}$$

gives $\|\xi_{k+1} - \xi_k\| \propto \|\bar{\boldsymbol{u}}(\xi)\|$, minimizing the path length is equivalent to minimizing the squared modulated velocity. The objective function is therefore

$$f(\rho_0,\, \sigma_0) = \bar{\boldsymbol{u}}^{\mathsf{T}}\bar{\boldsymbol{u}} = (\boldsymbol{M}\boldsymbol{u})^{\mathsf{T}}(\boldsymbol{M}\boldsymbol{u})\,, \tag{19}$$

and the optimization problem reads

$$\text{minimize}_{\rho_0,\,\sigma_0}\; f(\rho_0,\, \sigma_0) \qquad \text{subject to} \quad \rho_0,\, \sigma_0 \geq 0. \tag{20}$$

5.2 Expansion of the objective function

Expanding Eq. 19 using the structure of $\boldsymbol{M}$ yields

$$\begin{aligned} f = \bar{\boldsymbol{u}}^{\mathsf{T}}\bar{\boldsymbol{u}} &- \frac{c_1}{|\Gamma(\xi)|^{1/\rho}\, c_7} + \frac{c_2 + c_4}{|\Gamma(\xi)|^{1/\rho}\, \|\boldsymbol{t}\|\, \|\boldsymbol{n}\|} \\ &+ \frac{c_1}{|\Gamma(\xi)|^{2/\rho}\, c_7} + \frac{c_6}{|\Gamma(\xi)|^{2/\rho}\, \|\boldsymbol{t}\|\, \|\boldsymbol{n}\|}, \end{aligned} \tag{21}$$

with quadratic-form coefficients

$$c_1 = \boldsymbol{u}^{\mathsf{T}}\boldsymbol{n}\boldsymbol{n}^{\mathsf{T}}\boldsymbol{u}, \quad c_2 = \boldsymbol{u}^{\mathsf{T}}\boldsymbol{t}\boldsymbol{n}^{\mathsf{T}}\boldsymbol{u}, \quad c_4 = \boldsymbol{u}^{\mathsf{T}}\boldsymbol{n}\boldsymbol{t}^{\mathsf{T}}\boldsymbol{u},$$
$$c_6 = \boldsymbol{u}^{\mathsf{T}}\boldsymbol{n}\boldsymbol{t}^{\mathsf{T}}\boldsymbol{t}\boldsymbol{n}^{\mathsf{T}}\boldsymbol{u}, \quad c_7 = \boldsymbol{n}^{\mathsf{T}}\boldsymbol{n}.$$

5.3 Optimality conditions

The Karush–Kuhn–Tucker (KKT) conditions for (20) are

$$\nabla f(\boldsymbol{X}) + \boldsymbol{\mu} = \boldsymbol{0}, \qquad \boldsymbol{\mu} \leq \boldsymbol{0}, \tag{22}$$
$$\boldsymbol{\mu}^{\mathsf{T}}\boldsymbol{X} = 0, \qquad \boldsymbol{X} \geq \boldsymbol{0}, \tag{23}$$

where $\boldsymbol{X} = [\rho_0,\, \sigma_0]^{\mathsf{T}}$ and $\boldsymbol{\mu} = [\mu_1,\, \mu_2]^{\mathsf{T}}$. The gradient components are

$$\nabla f = \begin{bmatrix} \dfrac{k_1 k_2}{\rho_0^2}\, \dfrac{2c_1}{c_7\,|\Gamma|^{k_2/\rho_0}}\left(\dfrac{1}{|\Gamma|^{k_2/\rho_0}} - 1\right) \\[2ex] \dfrac{k_1 k_2}{\sigma_0^2}\, \dfrac{1}{\|\boldsymbol{n}\|\,\|\boldsymbol{t}\|\,|\Gamma|^{k_2/\sigma_0}}\left(c_2 + c_4 - \dfrac{2c_6}{\|\boldsymbol{n}\|\,\|\boldsymbol{t}\|\,|\Gamma|^{k_2/\sigma_0}}\right) \end{bmatrix}, \tag{24}$$

with $k_1 = \ln|\Gamma(\xi)|$ and $k_2 = \exp\!\left(\frac{1}{d_0\, d} - 1\right)$.

The system Eqs. 22–23 comprises three equations in four unknowns $(\rho_0, \sigma_0, \mu_1, \mu_2)$. An analytical solution can, in principle, be obtained by enumerating cases of the complementarity condition Eq. 23 (e.g. $\mu_2^* = \rho_0^* = 0$) and verifying the remaining KKT conditions. However, the algebraic complexity of Eq. 24 makes closed-form solutions impractical, and sufficient conditions for global optimality would still need to be checked. A numerical approach is therefore adopted: the Sequential Quadratic Programming (SQP) method and the interior-point method are employed to solve Eq. 20 [50, 51].

6 Three-dimensional Path-following

6.1 UAV Dynamics Model

A six-degree-of-freedom rigid-body model is adopted for the multirotor, neglecting rotor and propeller dynamics. The translational state is expressed in the inertial (North–East–Down) frame; the rotational state is expressed in the body-fixed frame. The equations of motion are

$$\dot{\boldsymbol{x}} = \boldsymbol{v}, \qquad \dot{\boldsymbol{v}} = g\,\boldsymbol{e}_3 - \frac{f}{m}\,R\,\boldsymbol{e}_3, \tag{25}$$
$$\dot{R} = R\,\hat{\boldsymbol{\Omega}}, \qquad \dot{\boldsymbol{\Omega}} = J^{-1}(\boldsymbol{M} - \boldsymbol{\Omega} \times J\boldsymbol{\Omega})\,, \tag{26}$$

where $\boldsymbol{x}, \boldsymbol{v} \in \mathbb{R}^3$ are the inertial position and velocity of the center of mass, $R \in SO(3)$ is the rotation from body to inertial frame, $\boldsymbol{\Omega} \in \mathbb{R}^3$ is the body angular velocity, $J = \mathrm{diag}(J_{xx}, J_{yy}, J_{zz})$ is the body-frame inertia, m is the vehicle mass, and $\boldsymbol{e}_3 = [0,\, 0,\, 1]^{\mathsf{T}}$. The hat map $\widehat{(\cdot)} : \mathbb{R}^3 \to \mathfrak{so}(3)$ denotes the skew-symmetric matrix such that $\hat{\boldsymbol{\Omega}}\,\boldsymbol{a} = \boldsymbol{\Omega} \times \boldsymbol{a}$, and its inverse is $(\cdot)^{\vee}$.

The control inputs are the scalar collective thrust $f = \sum_{i=1}^{4} f_i$ and the body-frame moment $\boldsymbol{M} \in \mathbb{R}^3$.

Control Allocation For a quadrotor in the cross (X) configuration with arm length ℓ and torque-to-thrust ratio $c_{\tau f}$, the mapping from rotor thrusts to control inputs is

$$\begin{bmatrix} f \\ M_1 \\ M_2 \\ M_3 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 & 1 & 1 \\ -\frac{\ell}{\sqrt{2}} & -\frac{\ell}{\sqrt{2}} & \frac{\ell}{\sqrt{2}} & \frac{\ell}{\sqrt{2}} \\ -\frac{\ell}{\sqrt{2}} & \frac{\ell}{\sqrt{2}} & \frac{\ell}{\sqrt{2}} & -\frac{\ell}{\sqrt{2}} \\ -c_{\tau f} & c_{\tau f} & -c_{\tau f} & c_{\tau f} \end{bmatrix}}_{\mathcal{M}} \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{bmatrix}, \tag{27}$$

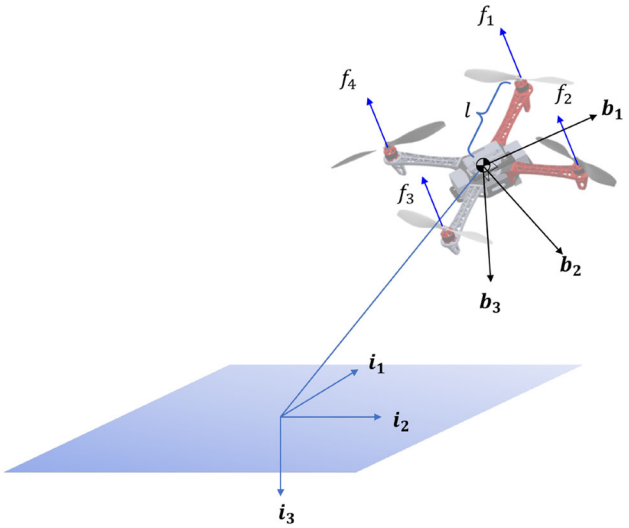


Fig. 11 Schematic of 3D UAV kinematics

where the sign pattern in the last row reflects counter-rotating rotor pairs. Individual rotor commands are recovered via $\boldsymbol{f}_{\text{rotor}} = \mathcal{M}^{-1}[f,\ \boldsymbol{M}^{\intercal}]^{\intercal}$. The model's schematic representation is illustrated in Fig. 11.

Coupling with IFDS Guidance The IFDS path planner of §3.2.1 supplies a kinematic reference path $\boldsymbol{x}_d(t)$ and a desired velocity $\dot{\boldsymbol{x}}_d(t)$. A two-loop structure closes the gap between this kinematic reference and the full dynamics Eqs. 25–26: the outer loop computes a desired thrust vector and heading from the translational error, and the inner loop generates $\boldsymbol{M}$ to drive R toward the corresponding desired attitude using the geometric SE(3) controller detailed in §6.2.

6.2 SE(3) Geometric Path-following

The IFDS algorithm of §3.2.1 produces, at each replanning instant, a discrete sequence of inertial-frame waypoints $\{\boldsymbol{W}_1, \boldsymbol{W}_2, \ldots, \boldsymbol{W}_N\} \subset \mathbb{R}^3$. To execute this path on the full nonlinear rigid-body dynamics of §6.1, we adopt the geometric tracking controller of Lee et al. [52, 53] on $SE(3)$. This controller is chosen because it is singularity-free on the rotation group and provides near-global exponential-tracking guarantees without the representational limitations of Euler-angle- or quaternion-based schemes.

Reference Trajectory The path is decomposed into independent segments $(\boldsymbol{W}_i, \boldsymbol{W}_{i+1})$. For each segment, let $\hat{\boldsymbol{d}} = (\boldsymbol{W}_{i+1} - \boldsymbol{W}_i)/\|\boldsymbol{W}_{i+1} - \boldsymbol{W}_i\|$ and $\psi_d = \operatorname{atan2}(\hat{d}_y, \hat{d}_x)$. A smooth reference is generated at constant cruise speed V_{ref}:

$$\boldsymbol{x}_d(t) = \boldsymbol{W}_i + \min\big(V_{\text{ref}}\, t,\ \|\boldsymbol{W}_{i+1} - \boldsymbol{W}_i\|\big)\,\hat{\boldsymbol{d}},$$
$$\dot{\boldsymbol{x}}_d = V_{\text{ref}}\,\hat{\boldsymbol{d}}, \qquad \boldsymbol{b}_{1d} = [\cos\psi_d,\ \sin\psi_d,\ 0]^{\intercal},$$

with all higher derivatives set to zero within a segment. This analytic parameterization avoids differentiating the noisy discrete waypoint stream; finite-difference filters are reserved solely for the measured velocity $\boldsymbol{v}$.

Control Law The translational errors $\boldsymbol{e}_x = \boldsymbol{x} - \boldsymbol{x}_d, \boldsymbol{e}_v = \boldsymbol{v} - \dot{\boldsymbol{x}}_d$ determine a desired thrust vector from which the desired rotation $R_c \in SO(3)$ is extracted following the construction in [52]. The geometric attitude and angular-velocity errors

$$\boldsymbol{e}_R = \tfrac{1}{2}\big(R_c^{\intercal} R - R^{\intercal} R_c\big)^{\vee}, \qquad \boldsymbol{e}_\Omega = \boldsymbol{\Omega} - R^{\intercal} R_c\, \boldsymbol{\Omega}_d,$$

then yield the scalar thrust f and body-frame moment $\boldsymbol{M}$ that drive the rigid-body dynamics. Full expressions for $(f, \boldsymbol{M})$ are given in [52, 53] and are implemented without modification. The equations of motion are integrated with a 4th-order Runge–Kutta scheme at step size T_s, and the rotation matrix R is re-projected onto $SO(3)$ after each step via singular-value decomposition to prevent numerical drift.

6.3 Waypoint-skipping

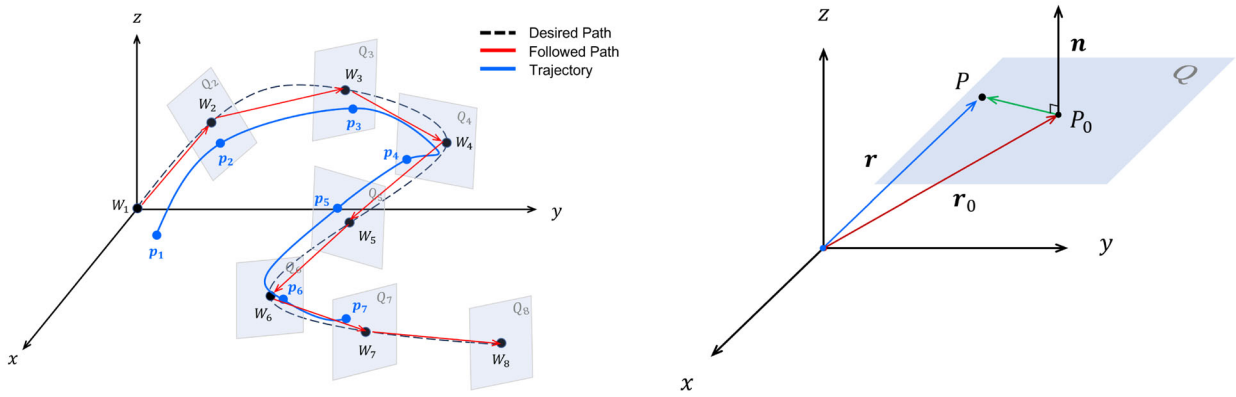
The segment $(\boldsymbol{W}_i, \boldsymbol{W}_{i+1})$ used in §6.2 is closed by a *waypoint-skipping* termination test that decides when the current segment has been completed, and the controller should advance to the next one. A path-following diagram is illustrated in Fig. 12a.

In practice, precise waypoint arrival within a specified time window is difficult due to inherent position errors. In some cases, the SE(3) tracker propagation may cause the UAV to pass the next waypoint before the standard arrival condition is satisfied. To address this issue, a waypoint-skipping logic is introduced as the termination condition for the segment-tracking inner loop in §6.2.

The method defines a plane perpendicular to the vector between two consecutive waypoints, allowing the algorithm to determine whether the UAV is ahead of or behind the target waypoint. If the UAV has already passed the current waypoint, the algorithm skips it and proceeds to the next waypoint rather than forcing the UAV to reverse direction. This prevents unnecessary back-and-forth motion and improves trajectory-tracking smoothness and efficiency. Consider the vectors $\boldsymbol{n} = \langle a,\ b,\ c\rangle$, $\boldsymbol{r} = \langle x,\ y,\ z\rangle$, and $\boldsymbol{r}_0 = \langle x_0,\ y_0,\ z_0\rangle$ as shown in Fig. 12b .
The vector $\overrightarrow{P_0P}$ on the plane Q can be expressed as $\overrightarrow{P_0P} = \boldsymbol{r} - \boldsymbol{r}_0$. Since $\boldsymbol{n}$ is perpendicular to $\overrightarrow{P_0P}$, we can write

$$\boldsymbol{n} \cdot (\boldsymbol{r} - \boldsymbol{r}_0) = 0$$
$$a(x - x_0) + b(y - y_0) + c(z - z_0) = 0 \tag{28}$$

Equation 28 is the equation of a plane Q given a point and a perpendicular normal vector, where the position of an arbi-



(a) Schematic of three-dimensional path-following

(b) A plane in 3D with a point and a perpendicular normal vector

Fig. 12 3D path following

trary point $(x,\ y,\ z)$ relative to the plane can be determined as follows:

1. If $a(x - x_0) + b(y - y_0) + c(z - z_0) = 0$, the point $(x,\ y,\ z)$ lies exactly on the plane;
2. If $a(x - x_0) + b(y - y_0) + c(z - z_0) > 0$, the point $(x,\ y,\ z)$ is in front of the plane (in the direction of the normal vector);
3. If $a(x - x_0) + b(y - y_0) + c(z - z_0) < 0$, the point $(x,\ y,\ z)$ is behind the plane (opposite to the normal vector).

For the path-following scenario, the problem reduces to finding the equation of the plane Q_{i+1} through the point $\boldsymbol{W}_{i+1}$ and perpendicular to the segment direction $\boldsymbol{R}_w = \boldsymbol{W}_{i+1} - \boldsymbol{W}_i = (x_{i+1} - x_i,\ y_{i+1} - y_i,\ z_{i+1} - z_i)^\mathsf{T}$. Expression Eq. 28 thus becomes

$$Q_{i+1}(x, y, z) = (x_{i+1} - x_i)(x - x_{i+1}) + (y_{i+1} - y_i)(y - y_{i+1}) + (z_{i+1} - z_i)(z - z_{i+1}). \tag{29}$$

The sign of $Q_{i+1}(x, y, z)$ in (29) determines whether the current UAV position is ahead of or behind the normal plane through the next waypoint: positive values indicate that the waypoint is still ahead, negative values indicate that the UAV has already passed it (and the waypoint is therefore skipped), and a zero value indicates that the UAV lies exactly on the plane. This waypoint-skipping scheme is illustrated in Fig. 13. In the example shown, waypoints $\boldsymbol{W}_{i+2}$, $\boldsymbol{W}_{i+4}$ and $\boldsymbol{W}_{i+5}$ are skipped.

7 Response to Dynamic Obstacles

7.1 Global and Local Path Adaptation

In this framework, two methods for generating the IFDS path are used: global and local. For the global path calculation, the initial and final waypoints were determined based on the

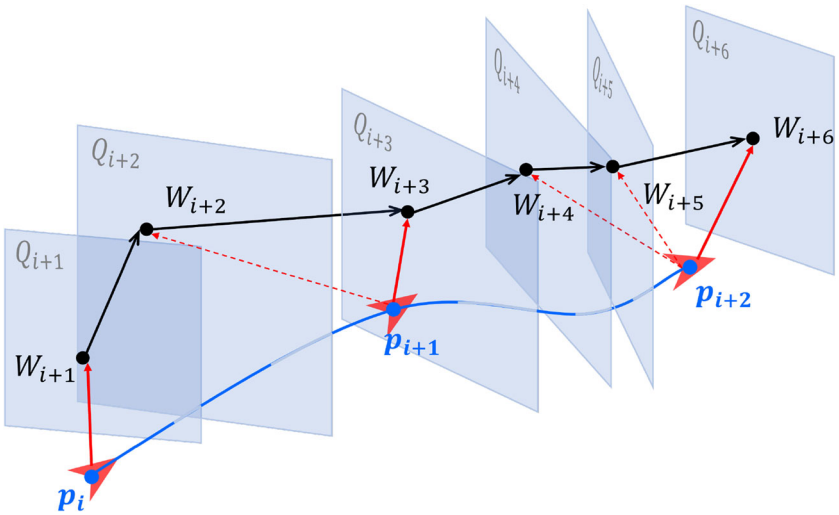


Fig. 13 Schematic of waypoint skipping for three-dimensional path-following

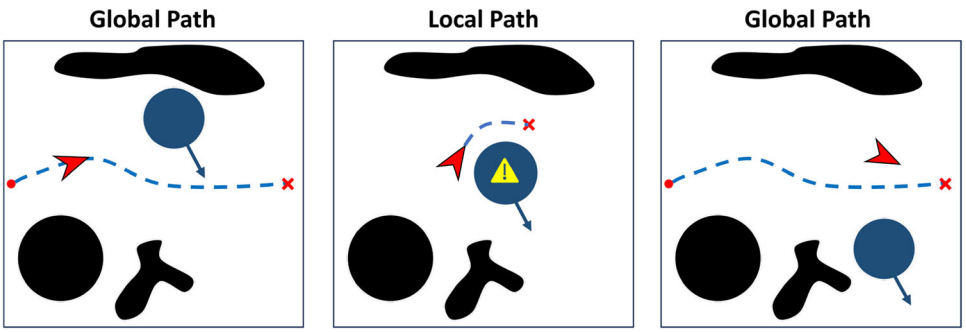


Fig. 14 Scenarios illustrating global and local path adaptation

starting location and the mission's destination. On the other hand, in local path planning, the path is generated based on the UAV's current position and propagates only a few steps ahead. This ensures a minimal computational load, as this path will be updated dynamically and used temporarily to avoid the dynamic obstacle.

Despite its longer computation time, the global path is calculated before the UAV's takeoff, thereby placing no extra load on the onboard processor during the flight. The local path, on the other hand, will be computed in real time on the UAV mid-flight. The ability to switch between the global and local paths allows the UAV to avoid unexpected obstacles without deviating significantly from the original route. A conceptual example of this is illustrated in Fig. 14.

7.1.1 Limitation of Global Path

The global path may not always be reliable in highly dynamic environments, such as during severe weather conditions. For instance, consider the scenario depicted in Fig. 15a . When constraints arise, the global path becomes unsuitable for generation from the starting location. However, a local path originating from the UAV's current position remains viable. This highlights the importance of transitioning to local path calculations when dynamic changes compromise the validity of the global path.

Additionally, Fig. 15b offers an alternative perspective. In this scenario, although the global path is valid, it may raise safety concerns or be inefficient for the UAV to follow.

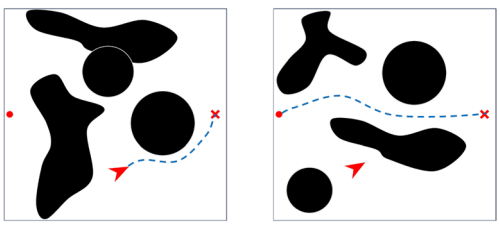


Fig. 15 Scenarios illustrating limitation of global path

In such cases, generating a local path based on the UAV's current position can enable more effective navigation.

7.2 Creeping Mode

Under nominal operation, the UAV follows the global path generated by the modified IFDS algorithm. If, however, the algorithm fails to produce a feasible path to the destination, the planner reverts to a *creeping mode*. In this mode, a short-horizon local path is adopted to advance the UAV into collision-free space while the speed controller simultaneously commands a deceleration toward hover. Normal operation resumes automatically once a feasible global path is re-established.

8 Implementation

8.1 Program Overview

The dynamic autorouting program consists of three main components: the IFDS algorithm, the path-optimizing algorithm, and the path following algorithm, all developed and simulated in MATLAB R2025b. The overall program flow is shown in Fig. 16.

The program initializes the global path pre-flight at the ground control station, then switches to the onboard computing system after takeoff. The onboard system operates through a series of checkpoints that trigger functions such as local path calculation, creeping mode, and path following.

8.2 UAV Models

The quadrotor UAV from [52, 54] is adopted in the MATLAB simulations, with the following mass and inertia properties:

$$J = [0.0820,\ 0.0845,\ 0.1377]\,\mathrm{kg\,m^2}, \quad m = 4.34\,\mathrm{kg},$$
$$d = 0.315\,\mathrm{m}, \quad c_{\tau f} = 8.004 \times 10^{-4}\,\mathrm{m}.$$

For the software-in-the-loop simulation, in which the ArduPilot controller replaces the SE(3) trajectory tracker, the

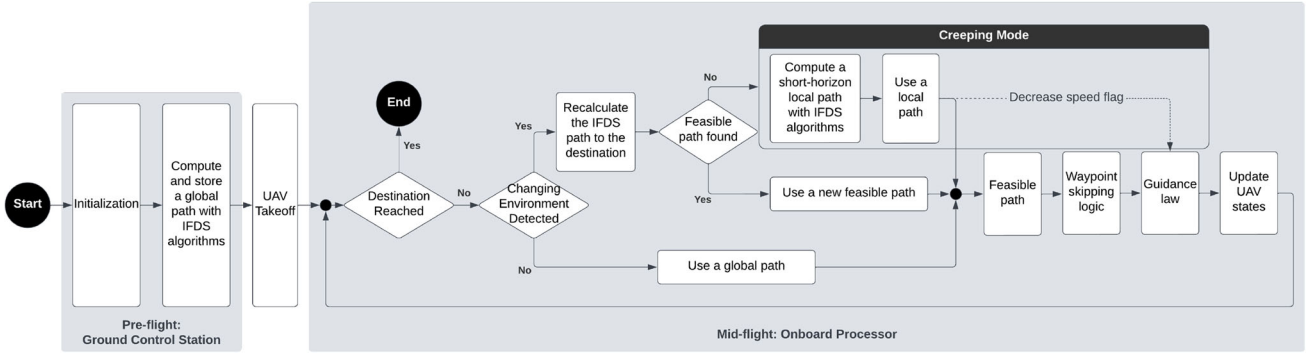


Fig. 16 Flowchart of the dynamic autorouting program

Iris quadcopter—the default SITL airframe of the ArduPilot–Gazebo simulator [55]—is adopted instead, with

$$J = [0.008,\ 0.015,\ 0.017]\ \mathrm{kg\,m^2}, \quad m = 1.50\ \mathrm{kg}.$$

The arm length and rotor torque-to-thrust ratio are embedded in the SDF model. In Gazebo, the `LiftDragPlugin` [56] computes the aerodynamic forces on each rotor blade from the parameters listed in Table 6.

8.3 Simulation Setups

We conducted a simulation study involving three scenarios that depict various static and dynamic obstacles of different shapes, as listed in Table 2. In all scenarios, the positions and characteristics of the obstacles are assumed to be known. The specific placement of obstacles for each scenario is detailed in Table 7. The positions of the dynamic obstacles change over time, following a sinusoidal pattern.

Table 3 outlines overall parameters used in the simulation. The simulated area encompasses a 200x200-square-meter region, with the UAV's cruising speed set at 10 m/s. This speed ensures that the UAV's responses to the changing environment are accurately reflected. The simulation duration ranges from 20 to 50 seconds, depending on how long it takes the UAV to reach the destination threshold. The initial IFDS repulsive parameter, ρ_0, is set between 1 and 2.5. This adjustment aims to achieve the most direct path possible. Meanwhile, the tangential parameter, σ_0, is set to 0.01 to control the lateral deviation of the path. This setting prioritizes longitudinal mobility, allowing the UAV to fly over obstacles.

8.4 Software-in-the-loop Demonstration

An additional software-in-the-loop (SITL) simulation is conducted to demonstrate the program's capability in a more practical setting, where noise, such as accelerometer and gyroscope noise, can be injected. Instead of a geometric SE(3) controller, the Ardupilot flight controller is adopted to follow the IFDS path. The simulation is visualized in Gazebo, where MAVLink messages are broadcast over the ROS2 network using MAVROS. In addition, MAVProxy is used as a light-weight ground control station program, and 3D trajectories are plotted in RViz2.

8.5 Noise Handling

Although sensor noise and state-estimation uncertainties are primarily addressed by complementary layers such as object

Table 2 Simulation scenario setups

Scenario	Type of environment	Type of obstacles	No. of obstacles	Description
1	Static	Static	12	This scenario includes only static conditions. The objective is to evaluate the performance of the global path generated by the program in the presence of fixed, fully known obstacles.
2	Dynamic	Static and dynamic	7	This scenario includes both static and dynamic obstacles within an environment that changes during flight. The aim is to evaluate the performance of the local path, including its creeping capabilities, and to compare computation times with and without optimization.
3	None	Static and dynamic	6	This scenario features both dynamic and static obstacles, but no environmental constraints. The adaptability of global and local paths will be examined in this context.

Table 3 Simulation parameters setups

Parameter	Variable name	Value
Simulation time	$rtsim$	20 - 50 s
Threshold for reaching destination	$targetThresh$	1 m
Starting location	$(X_i,\ Y_i,\ Z_i)$	$(0,\ 0,\ 0)\ m$
Target destination	$(X_f,\ Y_f,\ Z_f)$	$(200,\ 0,\ 50)\ m$
Desired aircraft cruising speed	C	$10\ m/s$
Initial aircraft location	$(x_i,\ y_i,\ z_i)$	$(0,\ -20,\ 5)\ m$
Initial aircraft yaw and pitch angle	$(\psi_i,\ \gamma_i)$	$(0,\ 0)\ rad$
Safeguarding distance	δ_g	10 m
Initial IFDS repulsive parameter	ρ_0	$1 - 2.5$
Initial IFDS tangential parameter	σ_0	0.01
IFDS Shape-following parameter	SF	0

localization, state-estimation filters, and low-level trajectory tracking, the proposed modified IFDS path planner incorporates an additional safeguard at the planning level. The safeguarding function, introduced in our previous work [32], augments the obstacle boundary by a buffer distance δ_g, ensuring a minimum clearance between the UAV and each obstacle. Formally, the modified obstacle function Γ_g is expanded from the original Γ such that the effective exclusion zone grows by δ_g in every direction. The buffer distance is expressed as a function of the IFDS tuning parameters and can be computed by aggregating uncertainties from multiple sources—for example, GPS position error, IMU drift, map discretization error, and contingency-maneuver distance—thereby providing a margin that scales with the expected uncertainty budget.

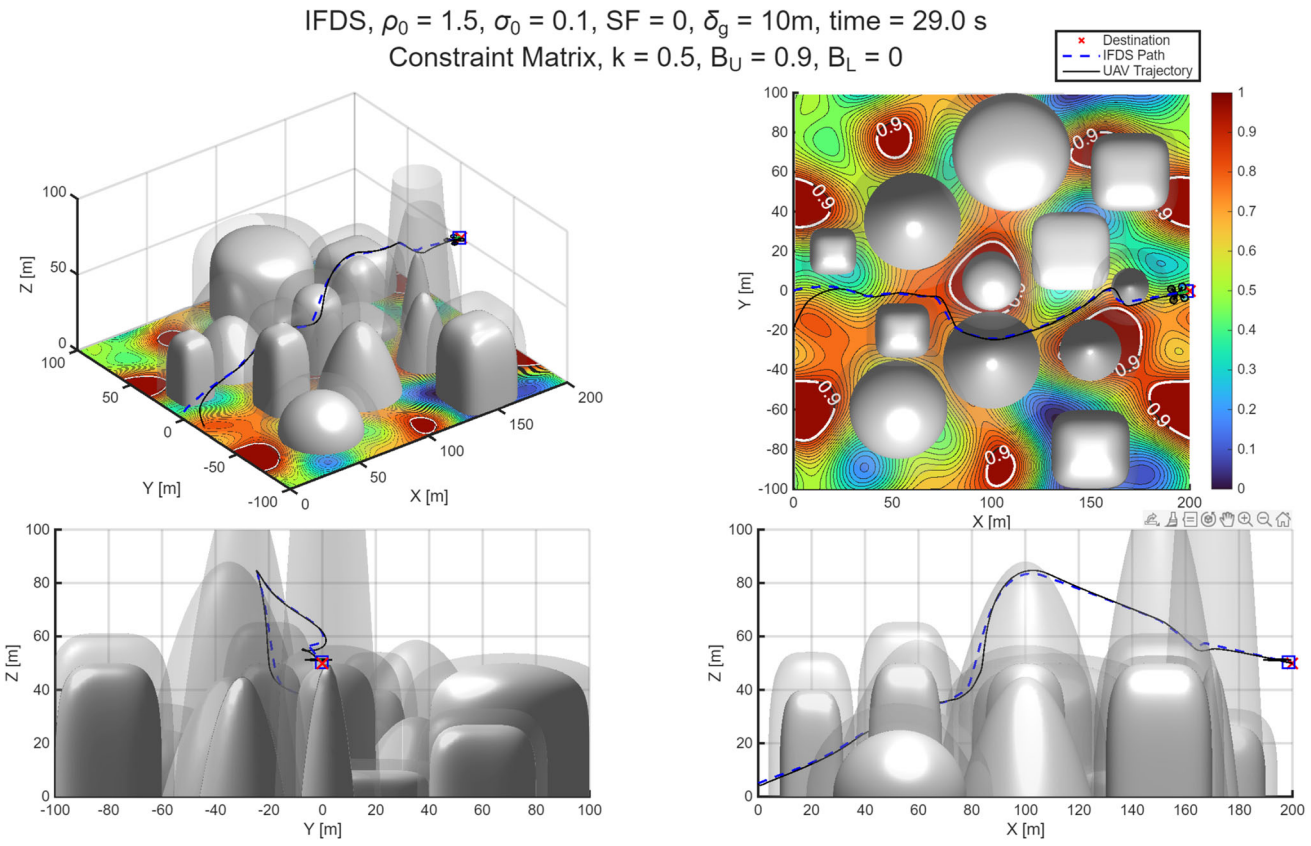


Fig. 17 Scenario 1: avoidance of multiple static obstacles in a static environment

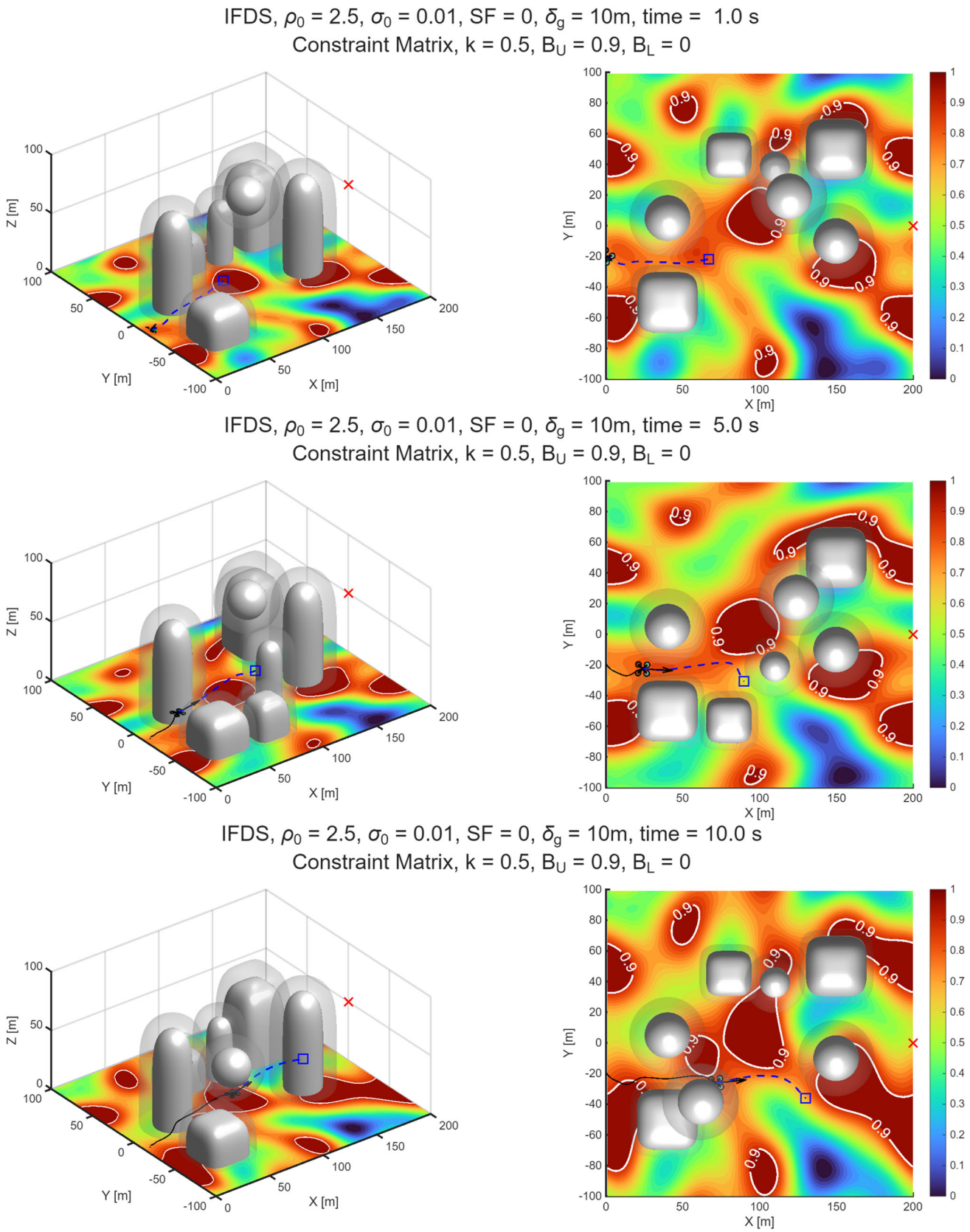


Fig. 18 Scenario 2: static and dynamic obstacles in dynamic environment avoidance, $t = 1 - 10\ s$ (non-optimized paths)

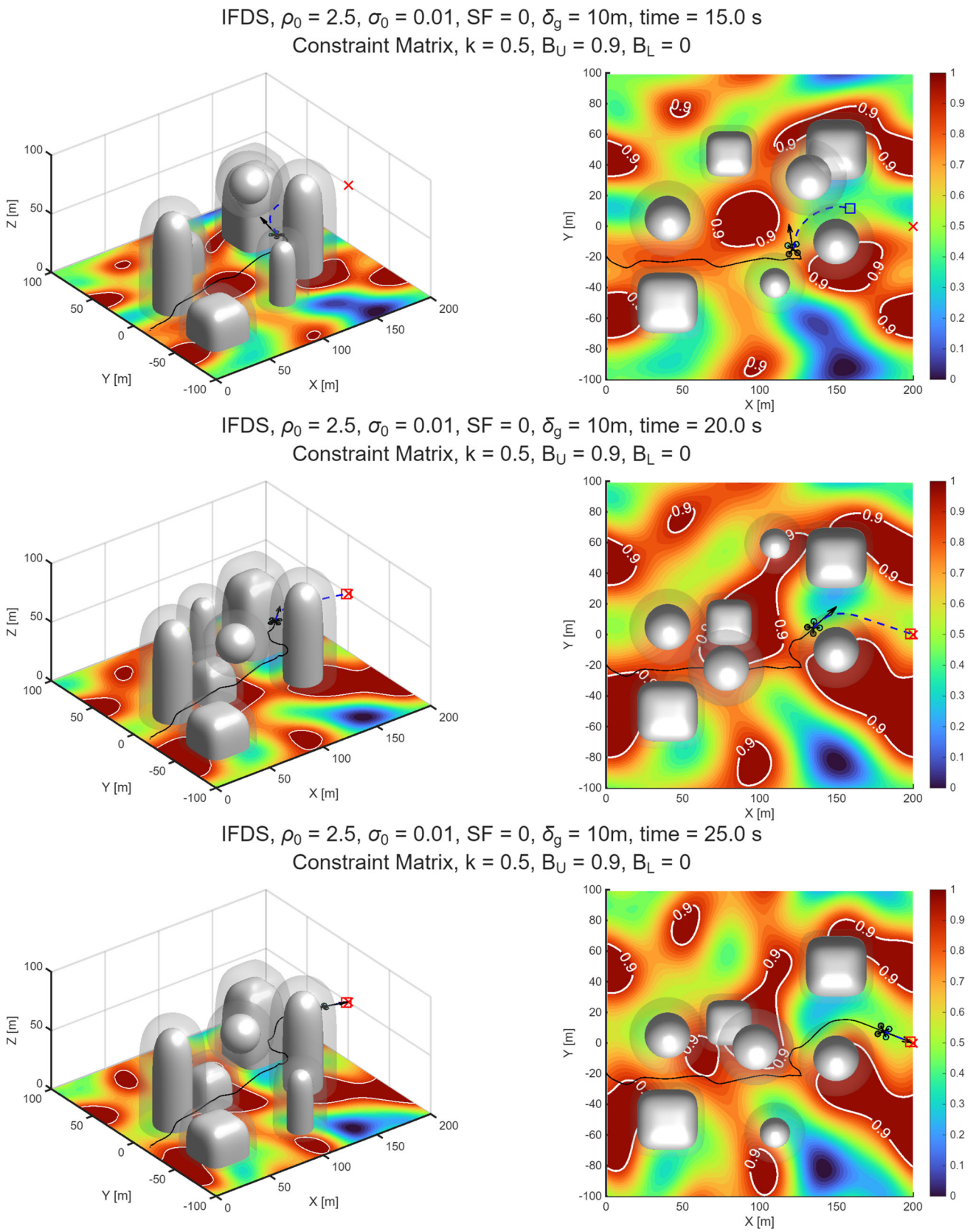


Fig. 19 Scenario 2: static and dynamic obstacles in dynamic environment avoidance, $t = 15 - 25\ s$ (non-optimized paths)

9 Results and Discussions

9.1 Global Path Planning in a Static Environment

Scenario 1 comprises multiple static obstacles and designated no-fly zones. Since the environment remains unchanged throughout the mission, the global path computed prior to departure is sufficient, as illustrated in Fig. 17. The UAV successfully navigates through tight gaps between buildings and no-fly zones, demonstrating the algorithm's capability in densely cluttered environments under static constraints.

This scenario highlights the importance of the path-following controller's tracking accuracy. In narrow passages, position-error margins are small; in practice, external disturbances such as wind gusts would further reduce these margins. Practical deployments should therefore incorporate appropriate safety buffers or feasibility checks before committing to tightly constrained maneuvers.

9.2 Local Path Planning in a Dynamic Environment

The results shown in Figs. 18 and 19 reveal a remarkable demonstration of the dynamic auto-routing capabilities in the dynamic environment of scenario 2. In a rapidly changing environment, the program adheres to the creeping mode. In this mode, local paths are constantly updated and gradually bring the UAV to a safe location closer to the destination until $t = 20\ s$, when a final clear path to the destination is obtained.

Figure 20 demonstrates successful tracking of the reference IFDS-generated path across all translational and rotational states throughout the mission. The UAV's velocity closely matches the commanded profile, with transient deviations occurring around $t \approx 13 - 17$ s, when the path undergoes sharp directional changes due to obstacle avoidance. These velocity excursions are bounded and recover within 1–2 s, indicating adequate bandwidth of the translational control loop. During steady cruise segments, the angular rates remain near zero, confirming minimal rotational effort when the reference IFDS-path is smooth. During the avoidance maneuvers at $t \approx 13 - 17$ s, brief angular rate spikes are observed as the attitude controller reorients the thrust vector to follow the abrupt velocity commands. Crucially, these transients remain within ± 15 rad/s and decay rapidly, demonstrating that the geometric SE(3) controller maintains stable attitude tracking even under aggressive path corrections without requiring explicit angular rate limiting in the guidance layer.

Figure 21 presents the position, velocity, and attitude tracking-error norms. During straight-line cruise ($t \approx 5 - 10$ s and $t > 22$ s), the position error stays below 1 m, and the velocity and attitude errors remain near zero. The peak errors

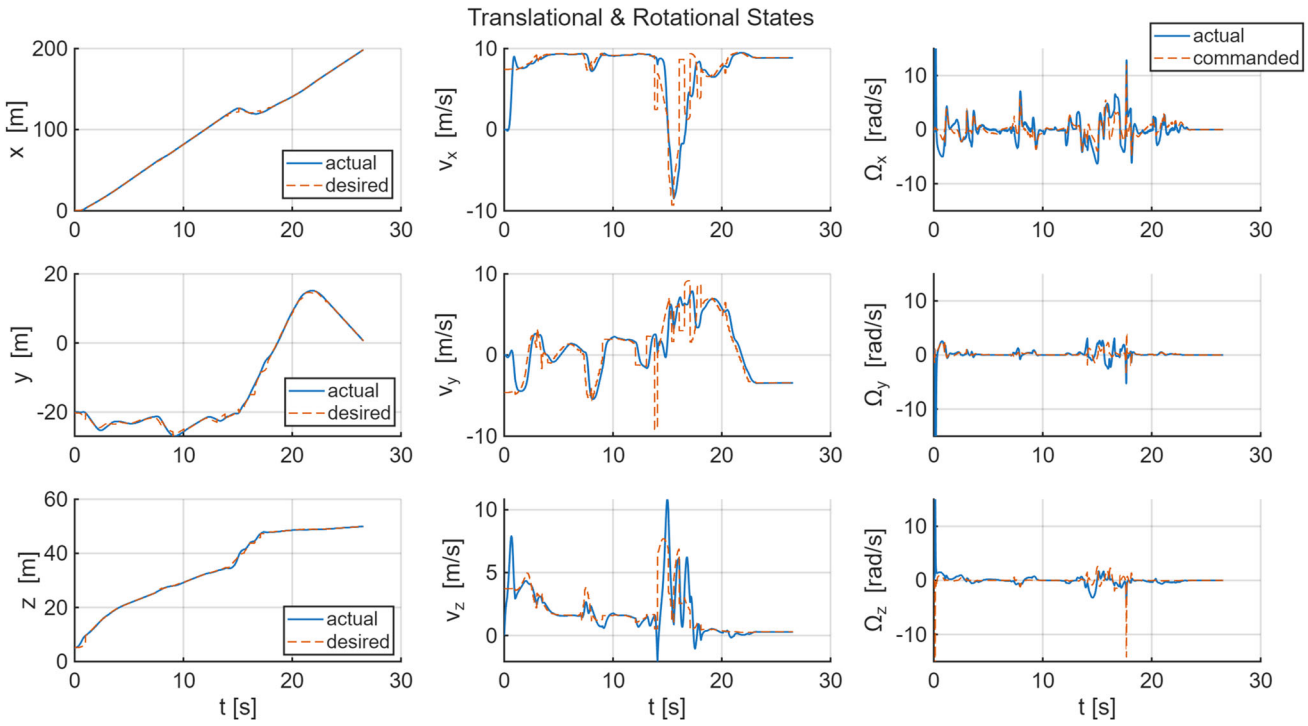


Fig. 20 Translational and rotational states of the simulated quadcopter in scenario 2

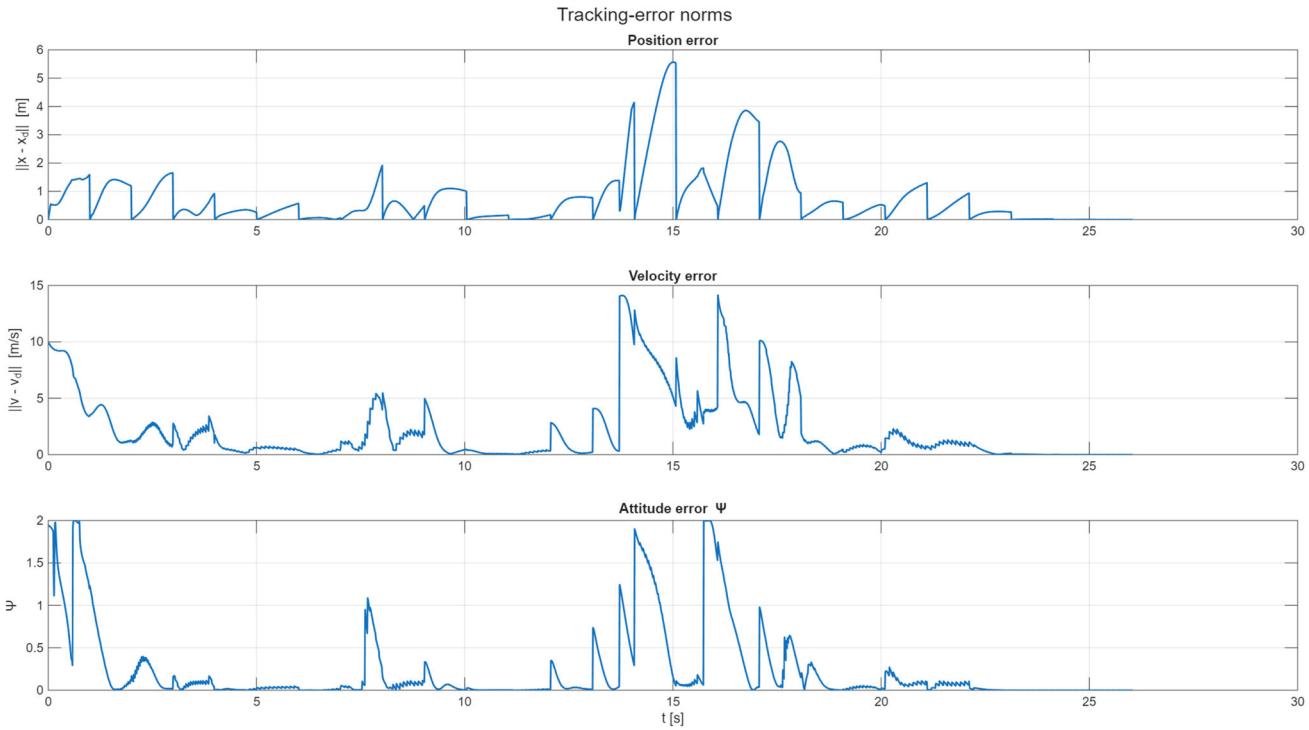


Fig. 21 Tracking error of the simulated quadcopter in scenario 2

occur during the obstacle-avoidance corridor ($t \approx 12 - 20$ s), where abrupt reference changes drive the position error to approximately 6 m, and the attitude error ψ briefly saturates near 2. All three signals recover within 1–2 s of each transient, confirming that the SE(3) controller remains stable under aggressive path corrections.

9.3 Global and local path planning for static and dynamic obstacles avoidance

In Scenario 3, environmental constraints were intentionally disregarded to evaluate the performance of the global and local path adaptation mechanisms exclusively. The results, as depicted in Figs. 22 and 23, demonstrate the effectiveness of these mechanisms. Initially, the UAV adhered to the global path. However, at $t = 10\ s$, the UAV detected an approaching dynamic obstacle. This detection triggered the program to switch to computing a local path, as shown by the red dashed line in the figures. This local path was updated every second until $t = 14\ s$ when the UAV successfully navigated past the dynamic obstacle. After $t = 15\ s$, no further interceptions were predicted, so the program reverted to following the global path. The UAV followed this path until it reached its final destination at $t = 30\ s$. Overall, the proposed global and local path adaptation mechanisms performed exceptionally well in this scenario, with negligible environmental factors. This method effectively deals with dynamic obstacles and significantly reduces the computational burden on the onboard processing unit.

9.4 Real-time Performance of Non-optimized Path

This section evaluates the real-time performance of the proposed dynamic autorouting algorithm for Scenario 2, where all computations are performed onboard. The simulations were conducted on a Windows 11 system equipped with an AMD Ryzen 9 processor (3.30 GHz) and 16 GB RAM using MATLAB R2025b. Each test was executed multiple times, and the average computation times were recorded.

Figure 25 illustrates the variation in computation time for local path planning in dynamic environments with different numbers of obstacles. The non-optimized IFDS local path achieved an average computation time of approximately 0.042 s, demonstrating the real-time capability of the proposed dynamic autorouting algorithm for onboard implementation. Furthermore, the computation time decreases to zero as the simulation approaches 26 s, indicating that the UAV has successfully reached its destination and no further local path updates are required.

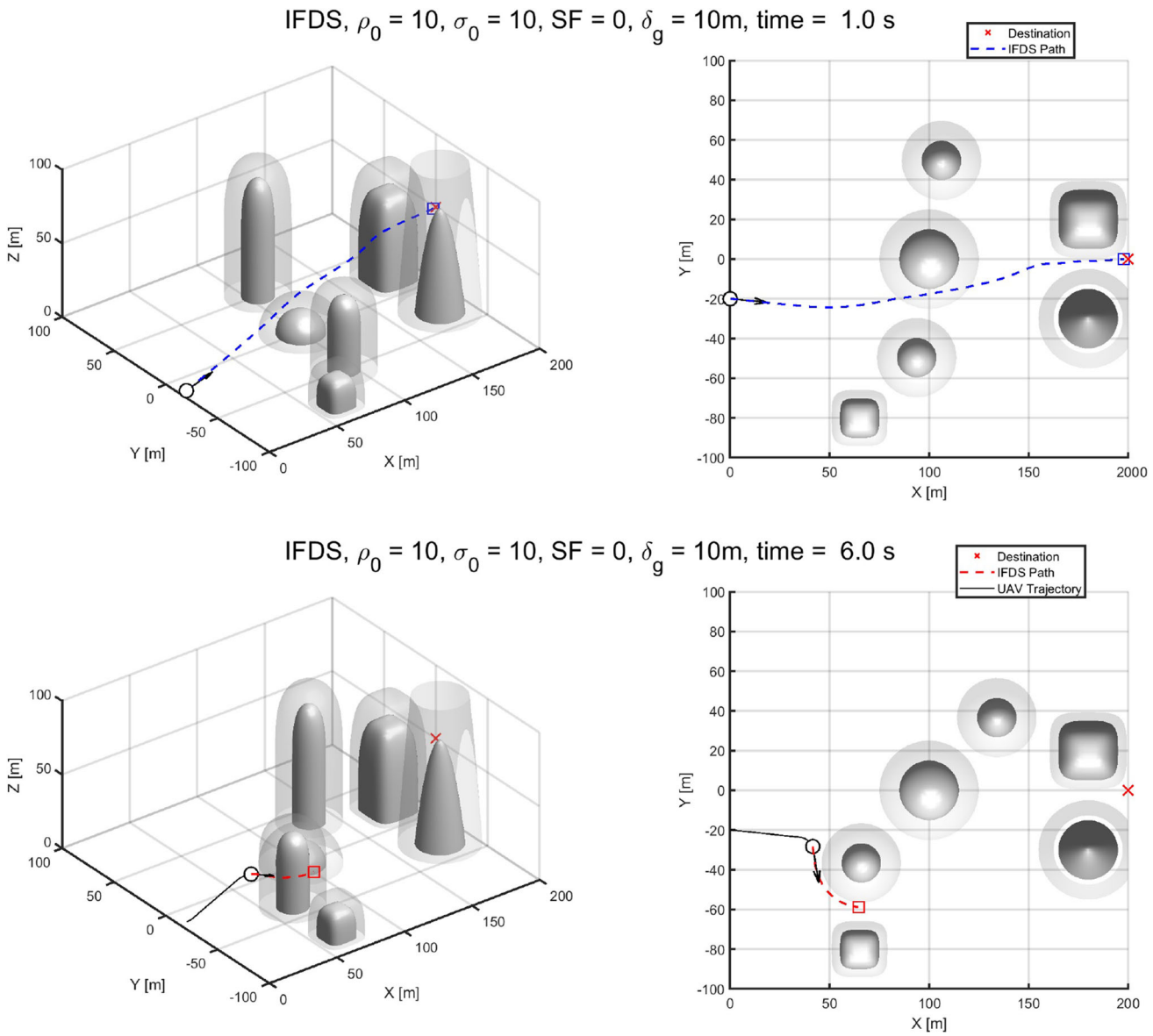


Fig. 22 Scenario 3: static and dynamic obstacles avoidance, $t = 5 - 10\ s$ (optimized paths)

9.5 Path Optimization Performance

9.5.1 Optimal Local Path

The performance of the optimized IFDS algorithm was evaluated for generating local paths in Scenario 2. The results indicate a trade-off between trajectory length and computational time. The optimized trajectory remains consistently close to the non-optimized trajectory, as illustrated in Fig. 24. The trajectory length for the optimized path is 239.88 m, compared to 246.10 m for the non-optimized path, representing an average reduction of 2.59%. Despite static and dynamic obstacles in the environment, the optimized IFDS algorithm reduces overall path length while maintaining feasible real-time computational cost, with an average execution time of 32.13 ms. These findings demonstrate that the proposed method generates near-optimal local trajectories in real time while preserving computational efficiency and effective obstacle avoidance.

Figure 25 provides a comparison between the computation times of the optimized and non-optimized methods. A detailed summary is presented in Table 4. From Fig. 25, it is noticeable that there are spikes in the computation time of the optimized IFDS at simulation time $t = 7s$ and $t = 15s$. This is due to the environment becoming overwhelmed and the emergence of numerous no-fly zones at these specific times. As a result, the optimization algorithm required more iterations to identify the optimal path, thereby increasing the

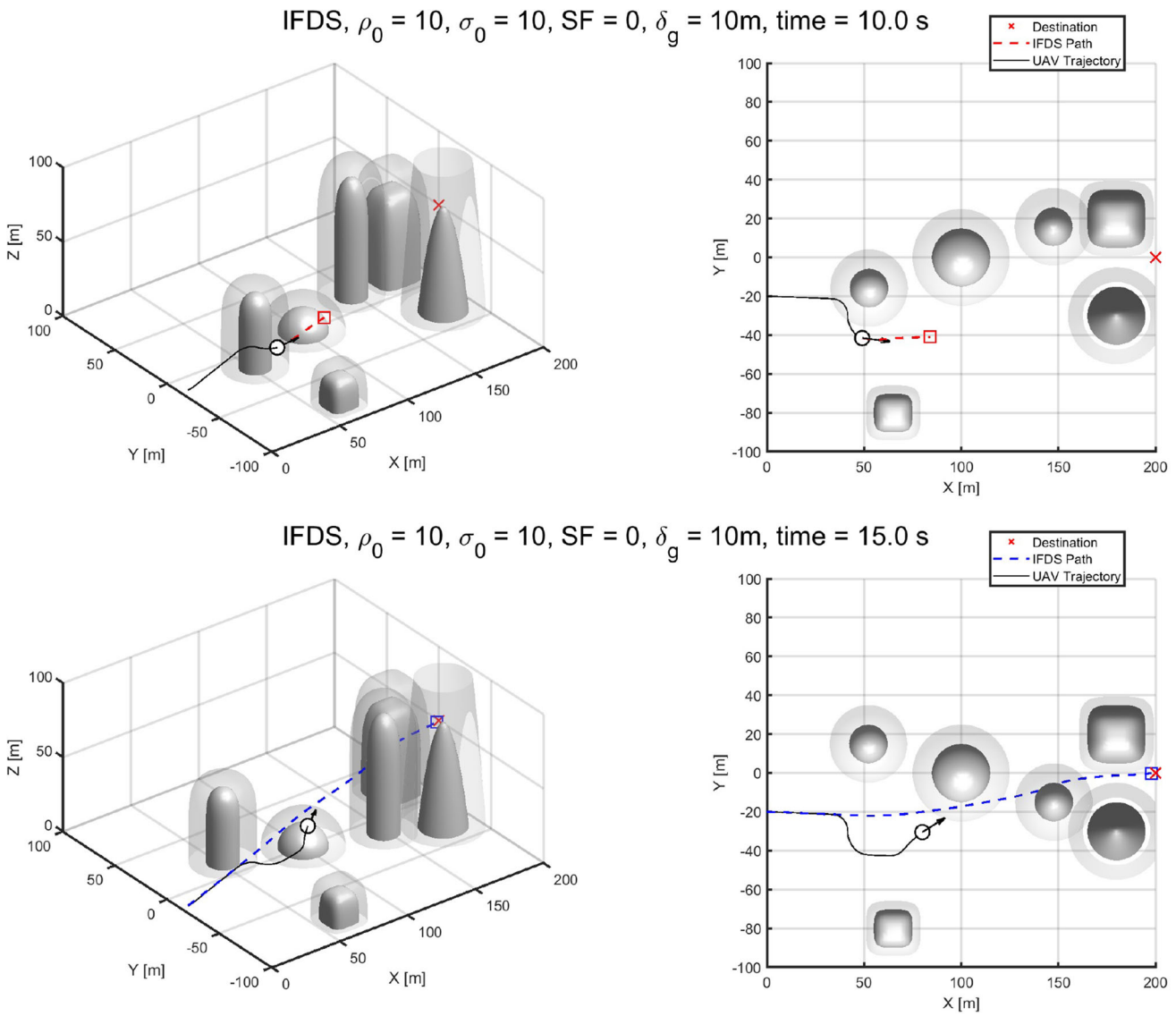


Fig. 23 Scenario 3: static and dynamic obstacles avoidance, $t = 15 - 30\ s$ (optimized paths)

computation time to 0.382 s and 0.323 s, respectively. Despite these spikes, the maximum computation time remains below one second, thereby meeting the real-time requirement.

9.5.2 Optimal Global Path

This section assesses the performance of global path optimization through a series of tests, including three scenarios: one obstacle, two obstacles, and twelve obstacles. Initially, these scenarios were computed with a high value of ρ_0 set to 10, resulting in extended redundant paths. Subsequently, the program computed these scenarios using either the interior-point method or sequential quadratic programming (SQP) to determine the optimal global path for each scenario. Table 5 details the resulting path lengths and computation times. In general, the interior-point method slightly outperforms SQP. Specifically, the interior-point method yields the highest distance reduction, resulting in the shortest path. However, it demands more iterations and computing time than SQP. It has been observed that the percentage reduction along the optimized path diminishes significantly as the number of obstacles increases.

In a scenario with only one obstacle, the optimized path is approximately 14.66% shorter, while in a scenario with twelve obstacles, the path length is reduced by only $1 - 2\%$. This phenomenon might be attributed to the fact that there

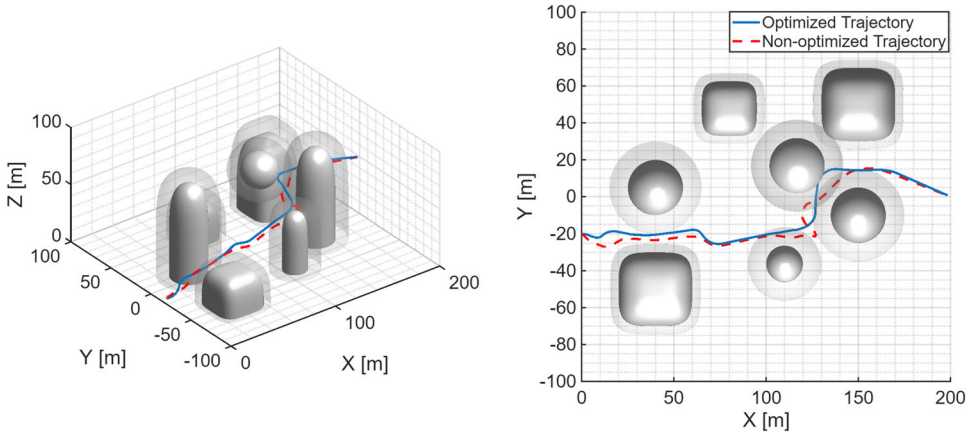


Fig. 24 Optimized and non-optimized trajectory in scenario 2

are fewer opportunities to find a shorter path in denser environments. Consequently, the original path derived from the IFDS algorithm tends to be close to optimal in more cluttered settings. The experiments show that the interior-point and SQP methods increase computing time by an average of 2000% and 651%, respectively. Comparatively, the SQP method exhibits moderately faster computation times than the interior-point method, as depicted in Fig. 26a.

For the interior-point method, the computation time reaches 2.13 seconds in a scenario with 12 obstacles. It is important to note that, in practice, the global path will be calculated at the ground control station with a powerful computer before the flight. Hence, this increased computation time does not burden the onboard processor during the flight.

9.5.3 Impact of the Number of Obstacles

Without optimization, an observable pattern emerges: the average computation time of the IFDS algorithm is linearly related to the number of objects in the area. This trend can be approximated by the equation $y = 0.0081x + 0.009$. It is worth noting that these scenarios are conducted within a $200 \times 200 \times 200$ meter space. By extrapolating from the trend depicted in Fig. 26b, it becomes apparent that the proposed algorithm retains its real-time computational capability for scenarios with up to approximately 122 obstacles within a $200 \times 200 \times 200$ meters space, given the hardware and setups used in this study. Furthermore, implementing static environmental constraints only marginally affects computational time, with a negligible increase at the millisecond scale. This reaffirms the viability and effectiveness of the proposed implementation of environmental constraints.

9.6 Initial Demonstration with Software-in-the-loop Simulation

To validate the IFDS planner under realistic actuation and sensing conditions, a dense urban environment is constructed in the Gazebo simulator and coupled to ArduCopter via the ArduPilot–Gazebo SITL bridge [55]. The Iris quadcopter is flown by the ArduPilot flight controller, which emits a synthetic but high-fidelity response that includes accelerometer and gyroscope noise, barometer drift, and airframe vibration coupled into the IMU. The IFDS planner runs as an off-board node, streaming waypoints to ArduCopter's `GUIDED` mode

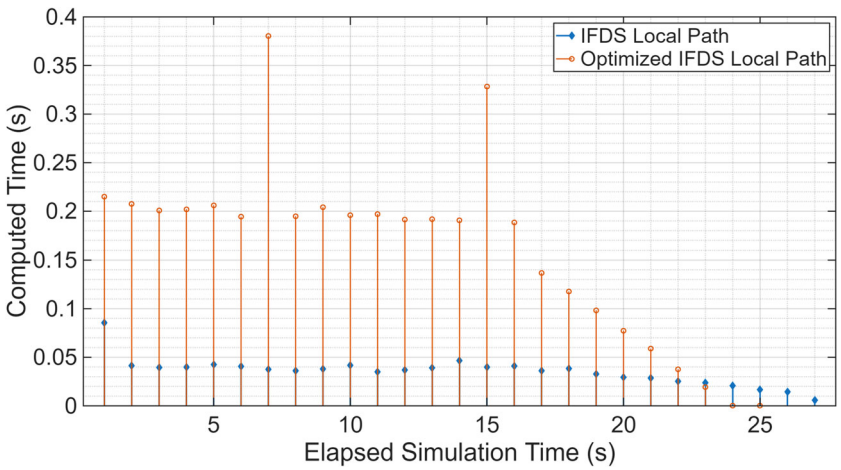


Fig. 25 Computed time for scenario 2: avoidance of static and dynamic obstacles in a dynamic environment

Table 4 Summary of computation time for optimized and non-optimized guidance in Scenario 2

Metric	Non-optimized (s)	Optimized (s)
Total steps	27	23
Average	0.035	0.175
Maximum	0.085	0.380
Minimum	0.0058	0.019

Table 5 Performance of optimized IFDS for global path-planning in various scenarios

No. of obstacles	Optimization method	Total path length (m)	Distance difference	Average computing time (ms)	Computing time difference	No. of iteration
1	None	248.21	–	13.3	–	–
1	Interior point	211.82	-14.66 %	317	$+2.28 \times 10^3$ %	12
1	SQP	214.79	-13.46 %	155	$+1.07 \times 10^3$ %	1
2	None	225.42	–	34.9	–	–
2	Interior point	213.39	-5.34 %	711	$+1.94 \times 10^3$ %	23
2	SQP	213.39	-5.34 %	162	+263.18 %	1
12	None	207.67	–	115	–	–
12	Interior point	204.08	-1.73 %	2130	$+1.75 \times 10^3$ %	6
12	SQP	205.81	-0.90 %	829	+620.87 %	8

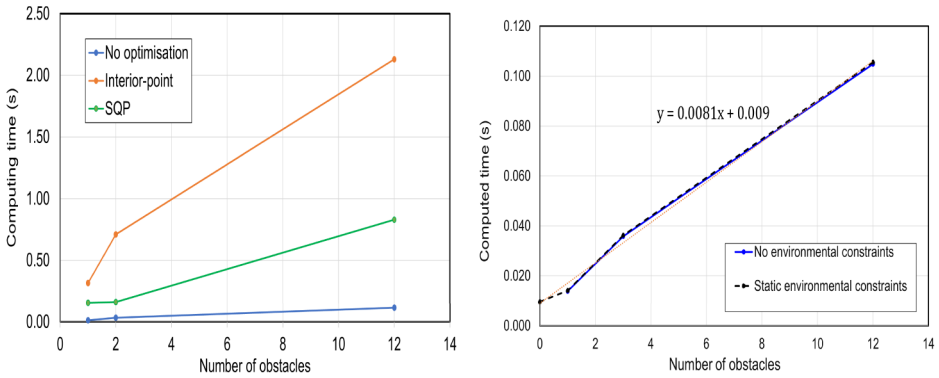


Fig. 26 Computation time for global path planning

(a) Computed time for various optimization methods in global path-planning

(b) Computed time for global path-planning with varying number of objects

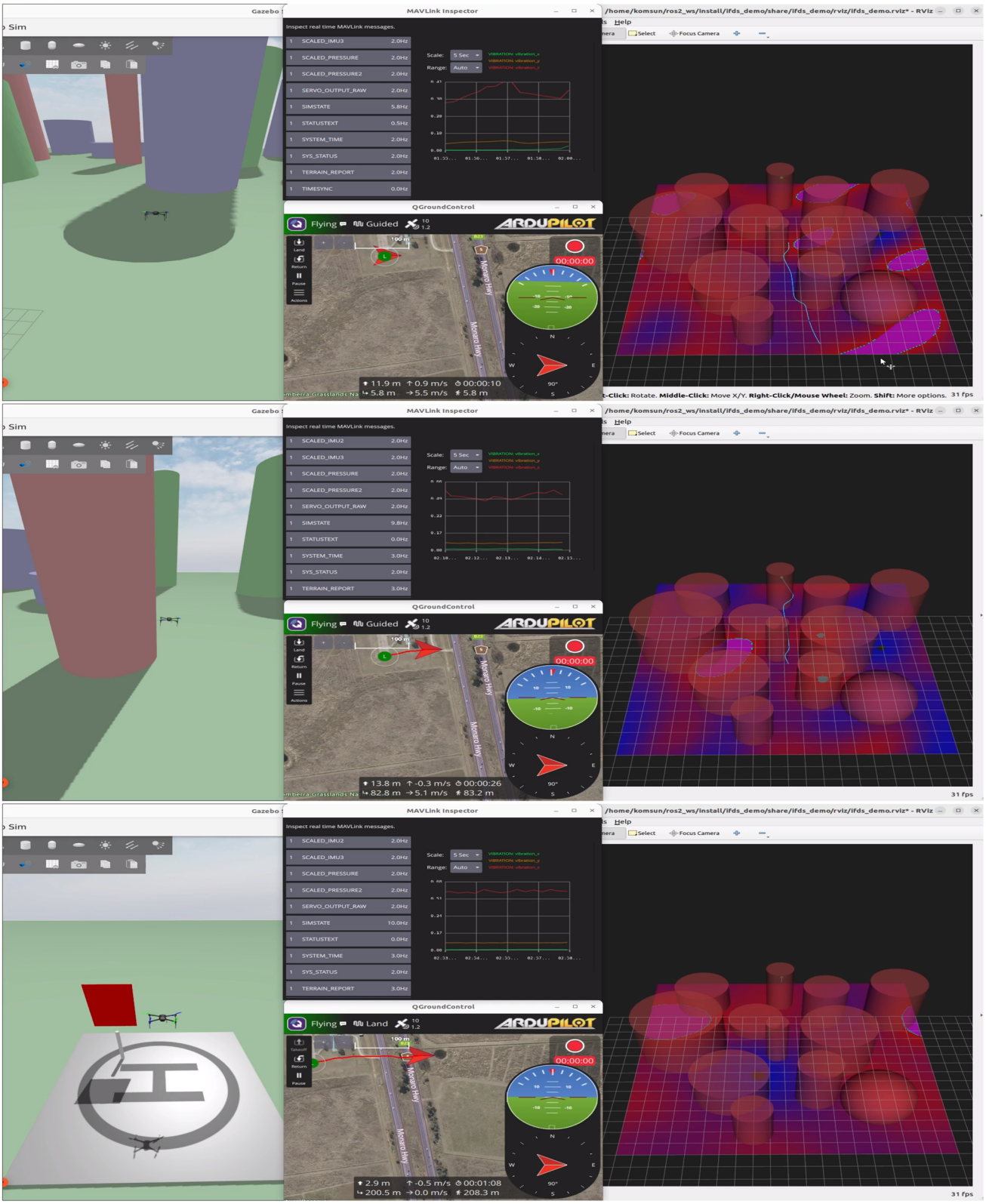


Fig. 27 Demonstration in the ArduPilot–Gazebo software-in-the-loop environment. **Left**: Gazebo simulator showing the Iris quadcopter in the urban scene; **center**: QGroundControl ground-station vibration plot and mission view; **right**: RViz2 visualizing the IFDS reference path streamed to ArduCopter

at 10 Hz while the cost map is updated online from the obstacle and weather fields.

Figure 27 shows a representative run in which the UAV successfully reroutes around the urban obstacles in a time-varying cost map. Despite the presence of sensor noise and vibration, the closed-loop trajectory remains collision-free and converges to the goal, confirming that the planner's robustness margins are preserved when the ideal tracker is replaced by a production autopilot.

Conclusion

This research focuses on developing dynamic path-planning methods for complex three-dimensional environments, including stationary and moving obstacles and changing environmental conditions. The proposed approach builds on the Interfered Fluid Dynamical System (IFDS) algorithms because of their high computational efficiency. Additionally, a 2D Fourier transform is used to construct a constraint matrix that models environmental conditions, including weather patterns, visibility, and temperature. A safety function is then implemented to maintain a safe, consistent distance between the path and object surfaces, regardless of the IFDS algorithm's tuning.

Based on this, we propose two path-planning modes for the UAV: global and local. When dynamic elements are detected in the scene, the path calculation automatically switches to the local path. We further optimize these paths to reduce their length using numerical methods, such as Sequential Quadratic Programming (SQP) and the interior-point method. The optimized results show a reduction in the global path length of $0.9-1.73\%$ compared to a non-optimized path in a tightly constrained scenario with 12 obstacles, while achieving up to a 14.66% reduction in path length in an open area with only one obstacle. For the optimized local path, there is an average reduction of approximately 2.59%, while maintaining real-time capability with an average compute time of 0.172 seconds.

Additional features are also proposed, including waypoint skipping and creeping logic. In situations with overwhelming environmental constraints or threats, the creeping mode is activated to guide the UAV to a safe area while getting closer to the destination. This mode remains active until the IFDS algorithm identifies a feasible path once the threats have subsided. This approach meets real-time requirements and significantly advances practical UAV applications.

Appendix A Algorithms and Listings

Table 6 Lift and drag parameters of the Iris quadcopter (per rotor blade) [55]

Parameter	Variable	Value
Initial angle of attack [rad]	`a0`	0.3
Slope of the lift curve below the stall angle [1/rad]	`cla`	4.2500
Slope of the lift curve above the stall angle [1/rad]	`cla_stall`	−0.025
Drag coefficient slope below the stall angle [1/rad]	`cda`	0.10
Drag coefficient slope above the stall angle [1/rad]	`cda_stall`	0.0
Pitching-moment coefficient slope below stall [1/rad]	`cma`	0.0
Pitching-moment coefficient slope above stall [1/rad]	`cma_stall`	0.0
Stall angle of attack [rad]	`alpha_stall`	1.4
Blade reference area [m^2]	`area`	0.002
Air density [kg/m^3]	`air_density`	1.2041
Center-of-pressure offset from blade root [m]	`cp`	[0.084, 0, 0]

Table 7 Obstacles setups in different scenarios

Scenario	Type of obstacle	Shape of obstacle	Origin of obstacle $(x_0,\ y_0,\ z_0)$	Properties of obstacle
1	Static	Cylinder	$(100,\ 5,\ 0)$	$r = 15\,m,\quad h = 50\,m$
	Static	Parallelepiped	$(140,\ 20,\ 0)$	$w = 20\,m,\quad h = 10\,m$
	Static	Parallelepiped	$(20,\ 20,\ 0)$	$w = 12\,m,\quad h = 40\,m$
	Static	Parallelepiped	$(55,\ -20,\ 0)$	$w = 14\,m,\quad h = 50\,m$
	Static	Parallelepiped	$(150,\ -80,\ 0)$	$w = 20\,m,\quad h = 50\,m$
	Static	Sphere	$(53,\ -60,\ 0)$	$r = 25\,m$
	Static	Cone	$(100,\ -35,\ 0)$	$r = 25\,m,\quad h = 45\,m$
	Static	Cone	$(170,\ 2,\ 0)$	$r = 10\,m,\quad h = 50\,m$
	Static	Cone	$(60,\ 35,\ 0)$	$r = 25\,m,\quad h = 30\,m$
	Static	Cylinder	$(110,\ 70,\ 0)$	$r = 30\,m,\quad h = 50\,m$
	Static	Parallelepiped	$(170,\ 60,\ 0)$	$w = 20\,m,\quad h = 27\,m$
	Static	Cone	$(150,\ -30,\ 0)$	$r = 16\,m,\quad h = 45\,m$
2	Static	Cylinder	$(40,\ 5,\ 0)$	$r = 15\,m,\quad h = 80\,m$
	Static	Cylinder	$(150,\ -10,\ 0)$	$r = 15\,m,\quad h = 80\,m$
	Static	Parallelepiped	$(40,\ -50,\ 0)$	$w = 25\,m,\quad h = 30\,m$
	Static	Parallelepiped	$(150,\ 50,\ 0)$	$w = 20\,m,\quad h = 60\,m$
	Dynamic	Cylinder	$(110,\ 60\sin(0.7t),\ 0)$	$r = 10\,m,\quad h = 50\,m$
	Dynamic	Parallelepiped	$(80,\ 60\cos(0.7t),\ 0)$	$w = 15\,m,\quad h = 30\,m$
	Dynamic	Sphere	$(100 + 40\sin(0.5t), 40\sin(0.5t),\ 60)$	$r = 15\,m$
3	Static	Sphere	$(100,\ 0,\ 0)$	$r = 15\,m$
	Static	Cone	$(180,\ -30,\ 0)$	$r = 15\,m,\quad h = 70\,m$
	Static	Parallelepiped	$(180,\ 20,\ 0)$	$w = 15\,m,\quad h = 60\,m$
	Static	Parallelepiped	$(65,\ -80,\ 0)$	$w = 10\,m,\quad h = 20\,m$
	Dynamic	Cylinder	$(100 + 50\sin(0.125t), 50\sin(0.125t),\ 0)$	$r = 10\,m,\quad h = 80\,m$
	Dynamic	Cylinder	$(100 - 50\sin(0.125t), -50\sin(0.125t),\ 0)$	$r = 10\,m,\quad h = 50\,m$

Algorithm 1 Constraint matrix generator.

procedure GENERATECONSTRAINTMATRIX($N_x, N_y, t_{max}, f,$ $numSeed$)
  Initialize: $A \leftarrow (3 Darray of size N_x \times N_y \times t_{max})$
  for $t \leftarrow 1$ to t_{max} **do**
    SeedRandom($numSeed$)
    $nx1 \leftarrow 1 + \textbf{floor}(0.999 \times f \times N_x)$
    $nx2 \leftarrow N_x - nx1 + 1$
    $ny1 \leftarrow 1 + \textbf{floor}(0.999 \times f \times N_y)$
    $ny2 \leftarrow N_y - ny1 + 1$
    $P \leftarrow$ **RandomMatrix**(N_x, N_y)
    $P \leftarrow$ **FFT2**(P)
    for $i \leftarrow nx1$ to $nx2 - 0.5\sin(t/2)$ **do**
      for $j \leftarrow 1$ to N_y **do**
        $P[i][j] \leftarrow 0$
      end for
    end for
    for $i \leftarrow 1$ to N_x **do**
      for $j \leftarrow ny1$ to $ny2 - 0.5\cos(t/2)$ **do**
        $P[i][j] \leftarrow 0$
      end for
    end for
    $P \leftarrow$ **iFFT2**(P)
    $P \leftarrow$ **NormalizedAndScale**(P)
    $A[t] \leftarrow P$
  end for
  return A
end procedure

Author Contributions K.T.: Conceptualization, derivation, simulation, writing manuscript; S.M.: Conceptualization, supervision, editing, result analysis; A.T.: Supervision, result analysis. All authors reviewed the manuscript.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Ethics approval and consent to participate Not applicable.

Consent for publication Not applicable.

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copy-

right holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Gutierrez-Martinez, M.A., Rojo-Rodriguez, E.G., Cabriales-Ramirez, L.E., Estabridis, K., Garcia-Salazar, O.: Genetic algorithm-based path planning of quadrotor uavs on a 3d environment. J. R. Aeronaut. Soc., pp. 1–37 (2024). https://doi.org/10.1017/aer.2024.132
2. Yang, T., Yang, F., Li, D.: A new autonomous method of drone path planning based on multiple strategies for avoiding obstacles with high speed and high density. Drones (2024). https://doi.org/10.3390/drones8050205
3. Wang, L., Liu, Y., Xu, W., Geng, C.: Deep reinforcement learning-based reactive trajectory planning method for uavs (2024). https://doi.org/10.1177/09544100241252233
4. Debnath, D., Hawary, A.F., Ramdan, M.I., Alvarez, F.V., Gonzalez, F.: Quicknav: An effective collision avoidance and path-planning algorithm for uas. Drones (2023). https://doi.org/10.3390/drones7110678
5. Castro, G.G.R., Berger, G.S., Cantieri, A.R., Teixeira, M., Lima, J., Pereira, A.I., Pinto, M.F.: Adaptive path planning for fusing rapidly exploring random trees and deep reinforcement learning in an agriculture dynamic environment uavs. Agriculture **13**, 354 (2023). https://doi.org/10.3390/agriculture13020354
6. Goerzen, C., Kong, Z., Mettler, B.: A survey of motion planning algorithms from the perspective of autonomous uav guidance. J. Intell. Rob. Syst. **57**(1), 65–100 (2010)
7. Lindqvist, B., Mansouri, S.S., Haluška, J., Nikolakopoulos, G.: Reactive navigation of an unmanned aerial vehicle with perception-based obstacle avoidance constraints. IEEE Trans. Control Syst. Technol. **30**(5), 1847–1862 (2021)
8. Liu, Y., Nejat, G.: Robotic urban search and rescue: A survey from the control perspective. J. Intell. Robot. Syst. **72**(2), 147–165 (2013)
9. Zhong, X., Tian, J., Hu, H., Peng, X.: Hybrid path planning based on safe a* algorithm and adaptive window approach for mobile robot in large-scale dynamic environment. J. Intell. Robot. Syst. **99**(1), 65–77 (2020)
10. Tzafestas, S.G.: Mobile robot control and navigation: A global overview. J. Intell. Robot. Syst. **91**(1), 35–58 (2018)
11. C. Vangasse, A., JR Freitas, E., V. Raffo, G., CA Pimenta, L.: Safe navigation on path-following tasks: a study of mpc-based collision avoidance schemes in distributed robot systems. J. Intell. Robot. Syst. **110**(4), 166 (2024)
12. Tao, J.-w., Ji, W.-c., Fan, Q.-j.: An effective approach of collision avoidance for uav. J. Intell. Robot. Syst. **108**(2), 18 (2023)
13. Yin, C., Xiao, Z., Cao, X., Xi, X., Yang, P., Wu, D.: Offline and online search: Uav multiobjective path planning under dynamic urban environment. IEEE Internet Things J. **5**, 546–558 (2018). https://doi.org/10.1109/JIOT.2017.2717078
14. Wu, C., Huang, X., Luo, Y., Leng, S., Wu, F.: An improved sparse hierarchical lazy theta* algorithm for uav real-time path planning in unknown three-dimensional environment. In: 2020 IEEE 20th International Conference on Communication Technology (ICCT), pp. 673–677 (2020). https://doi.org/10.1109/ICCT50939.2020.9295690
15. Elhoseny, M., Tharwat, A., Hassanien, A.E.: Bezier curve based path planning in a dynamic field using modified genetic algorithm. J. Comput. Sci. **25**, 339–350 (2018). https://doi.org/10.1016/j.jocs.2017.08.004
16. Shao, S., Peng, Y., He, C., Du, Y.: Efficient path planning for uav formation via comprehensively improved particle swarm optimization. ISA Trans. **97**, 415–430 (2020). https://doi.org/10.1016/j.isatra.2019.08.018
17. Jamshidi, V., Nekoukar, V., Refan, M.H.: Real time uav path planning by parallel grey wolf optimization with align coefficient on can bus. Clust. Comput. **24**, 2495–2509 (2021). https://doi.org/10.1007/s10586-021-03276-6
18. Ding, Q., Xu, X.: Improved GWO Algorithm for UAV Path Planning on Crop Pest Monitoring. Int. J. Interact. Multim. Artif. Intell. **7**, 5 (2022). https://doi.org/10.9781/ijimai.2022.07.002
19. Lui, X., Long, Y., Li, T., Huang, T.: 3D Path Planning of Unmanned Aerial Vehicle Based on Improved Grey Wolf Optimization Algorithm. In: 2022 China Automation Congress (CAC), pp. 4285–4290 (2022). https://doi.org/10.1109/CAC57257.2022.10055505
20. Subies Hueso, J., Mondal, S., Tsourdos, A., Chadwick, A.: Real-time collision avoidance trajectory planner using generalized vector explicit guidance. In: AIAA SciTech 2023 Forum, p. 1734 (2023)
21. Ohlmeyer, E.J., Phillips, C.A.: Generalized vector explicit guidance. J. Guid. Control. Dyn. **29**(2), 261–268 (2006). https://doi.org/10.2514/1.14956
22. Tordesillas, J., How, J.P.: Mader: Trajectory planner in multiagent and dynamic environments. IEEE Trans. Rob. **38**(1), 463–476 (2021)
23. Kondo, K., Figueroa, R., Rached, J., Tordesillas, J., Lusk, P.C., How, J.P.: Robust mader: Decentralized multiagent trajectory planner robust to communication delay in dynamic environments. IEEE Robot. Autom. Lett. (2023)
24. Tordesillas, J., How, J.P.: Minvo basis: Finding simplexes with minimum volume enclosing polynomial curves. Comput. Aided Des. **151**(C) (2022). https://doi.org/10.1016/j.cad.2022.103341
25. Zhou, B., Gao, F., Pan, J., Shen, S.: Robust real-time uav replanning using guided gradient-based optimization and topological paths (2019). arXiv:1912.12644 arXiv preprint
26. Zhou, B., Gao, F., Wang, L., Liu, C., Shen, S.: Robust and efficient quadrotor trajectory generation for fast autonomous flight. IEEE Robot. Autom. Lett. **4**(4), 3529–3536 (2019)
27. Zhou, B., Pan, J., Gao, F., Shen, S.: Raptor: Robust and perception-aware trajectory replanning for quadrotor fast flight (2020). arXiv:2007.03465 arXiv preprint
28. Xie, C., Li, S., Qin, X., Fu, S., Zhang, X.: Multiple elite strategy enhanced rime algorithm for 3d uav path planning. Sci. Rep. **14**(1), 21734 (2024)
29. Han, L., Gao, F., Zhou, B., Shen, S.: FIESTA: Fast Incremental Euclidean Distance Fields for Online Motion Planning of Aerial Robots (2019). arxiv.org/abs/1903.02144
30. Zhou, X., Wang, Z., Ye, H., Xu, C., Gao, F.: Ego-planner: An esdf-free gradient-based local planner for quadrotors. IEEE Robot. Autom. Lett. **6**(2), 478–485 (2021). https://doi.org/10.1109/LRA.2020.3047728
31. Zong, Z., Li, D., Dong, X., Cui, Y., Yang, B., Xiang, J., Tu, Z.: Risk-aware enabled path planning for drones flight in unknown environment. J. Intell. Robot. Syst. **111**(2), 47 (2025)
32. Tamanakijprasart, K., Mondal, S., Tsourdos, A.: Dynamic path planning of uav in three-dimensional complex environment based on interfered fluid dynamical system. In: AIAA SCITECH 2024 Forum, p. 2091 (2024)
33. Yao, P., Wang, H., Su, Z.: Uav feasible path planning based on disturbed fluid and trajectory propagation. Chin. J. Aeronaut. **28**, 1163–1177 (2015). https://doi.org/10.1016/J.CJA.2015.06.014
34. Yao, P., Wang, H., Su, Z.: Real-time path planning of unmanned aerial vehicle for target tracking and obstacle avoidance in complex dynamic environment. Aerosp. Sci. Technol. **47**, 269–279 (2015). https://doi.org/10.1016/J.AST.2015.09.037

35. Wang, Y., Wang, H., Wen, J., Lun, Y., Wu, J.: Obstacle avoidance of uav based on neural networks and interfered fluid dynamical system. In: 2020 3rd International Conference on Unmanned Systems (ICUS), pp. 1066–1071 (2020). https://doi.org/10.1109/ICUS50048.2020.9274988
36. Yao, P., Zhao, S.: Three-dimensional path planning for auv based on interfered fluid dynamical system under ocean current (june 2018). IEEE Access **6**, 42914–42916 (2018). https://doi.org/10.1109/ACCESS.2018.2861468
37. Celestini, D., Primatesta, S., Capello, E.: Trajectory planning for uavs based on interfered fluid dynamical system and bézier curves. IEEE Robot. Autom. Lett. **7**, 9620–9626 (2022). https://doi.org/10.1109/LRA.2022.3191855
38. Zhang, Y., Wang, H.: Adaptive interfered fluid dynamic system algorithm based on deep reinforcement learning framework. Lecture Notes in Electrical Engineering 861 LNEE, pp. 1388–1397 (2022). https://doi.org/10.1007/978-981-16-9492-9_139/TABLES/1
39. Fan, S., Chen, M.: Route planning of agricultural plant protection unmanned helicopter based on interfered fluid dynamical system. In: Proceedings - 9th International Conference on Intelligent Human-Machine Systems and Cybernetics, IHMSC 2017 **2**, 114–117 (2017). https://doi.org/10.1109/IHMSC.2017.141
40. Wu, J., Wang, H., Li, N., Su, Z.: Formation obstacle avoidance: A fluid-based solution. IEEE Syst. J. **14**, 1479–1490 (2020). https://doi.org/10.1109/JSYST.2019.2917786
41. Wu, J., Wang, H., Liu, Y., Zhang, M., Wu, T.: Learning-based fixed-wing uav reactive maneuver control for obstacle avoidance. Aerosp. Sci. Technol. **126**, 107623 (2022). https://doi.org/10.1016/j.ast.2022.107623
42. Zhang, X., Lu, X., Jia, S., Li, X.: A novel phase angle-encoded fruit fly optimization algorithm with mutation adaptation mechanism applied to uav path planning. Appl. Soft Comput. **70**, 371–388 (2018)
43. Zhang, X., Duan, H.: An improved constrained differential evolution algorithm for unmanned aerial vehicle global route planning. Appl. Soft Comput. **26**, 270–284 (2015)
44. Zhang, X., Xia, S., Li, X., Zhang, T.: Multi-objective particle swarm optimization with multi-mode collaboration based on reinforcement learning for path planning of unmanned air vehicles. Knowl.-Based Syst. **250**, 109075 (2022)
45. Yu, X., Li, C., Zhou, J.: A constrained differential evolution algorithm to solve uav path planning in disaster scenarios. Knowl.-Based Syst. **204**, 106209 (2020)
46. Wang, J., Li, Y., Li, R., Chen, H., Chu, K.: Trajectory planning for uav navigation in dynamic environments with matrix alignment dijkstra. Soft. Comput. **26**(22), 12599–12610 (2022)
47. Yan, C., Xiang, X., Wang, C.: Towards real-time path planning through deep reinforcement learning for a uav in dynamic environments. J. Intell. Robot. Syst. **98**(2), 297–309 (2020)
48. Wang, H., Lyu, W., Yao, P., Liang, X., Liu, C.: Three-dimensional path planning for unmanned aerial vehicle based on interfered fluid dynamical system. Chin. J. Aeronaut. **28**, 229–239 (2015). https://doi.org/10.1016/J.CJA.2014.12.031
49. Khalil, H.K.: Nonlinear Systems, Prentice Hall (2002). (3rd edn)
50. Bazaraa, M.S., Sherali, H.D., Shetty, C.M.: Nonlinear Programming: Theory and Algorithms. Wiley (2013). https://books.google.co.uk/books?id=nDYz-NIpIuEC
51. Betts, J.T.: Practical Methods for Optimal Control and Estimation Using Nonlinear Programming, 2nd Edn. Society for Industrial and Applied Mathematics (2010). https://doi.org/10.1137/1.9780898718577, https://epubs.siam.org/doi/abs/10.1137/1.9780898718577
52. Lee, T., Leok, M., McClamroch, N.H.: Geometric tracking control of a quadrotor uav on se(3). In: 49th IEEE Conference on Decision and Control (CDC), pp. 5420–5425 (2010). https://doi.org/10.1109/CDC.2010.5717652
53. Lee, T., Leok, M., McClamroch, N.H.: Control of Complex Maneuvers for a Quadrotor UAV using Geometric Methods on SE(3) (2011). arxiv.org/abs/1003.2005
54. Pounds, P., Mahony, R., Corke, P.: Modelling and control of a large quadrotor robot. Control. Eng. Pract. **18**(7), 691–699 (2010)
55. ArduPilot Development Team: ardupilot_gazebo: Gazebo Plugin for ArduPilot Software-in-the-Loop Simulation (2024). https://github.com/ArduPilot/ardupilot_gazebo. Accessed 22 May 2026
56. OSRF: Gazebo Aerodynamics (2015). https://classic.gazebosim.org/tutorials?tut=aerodynamics

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Komsun Tamanakijprasart received his BEng in Mechanical and Aerospace Engineering from Kyushu University, Japan, in 2022, and his MSc in Autonomous Vehicle Dynamics and Control from Cranfi eld University, UK, in 2023. He is currently pursuing a PhD in Aerospace Engineering at Cranfield University. His research interests include autonomous guidance andnavigation for uncrewed aircraft systems, path planning, optimization, and multi-agent systems coordination.

Sabyasachi Mondal received the PhD degree in Aerospace Engineering from Indian Institute of Science, Bangalore, India.He is currently working as a lecturer at Cranfi eld University. His research interests include Guidance of Aerospace Vehicles,Control of Multi-Agent systems, Autonomous Platform design, and Path Planning.

Antonios Tsourdos obtained an MEng on Electronic, Control and Systems Engineering from the University of Sheffi eld(1995), an MSc on Systems Engineering from Cardiff University (1996) and a PhD on Nonlinear Robust Flight Control Designand Analysis from Cranfi eld University (1999). He is a Professor of Autonomous Systems and Control with Cranfi eld Universityand Head of the Centre of Autonomous and Cyber-Physical Centre. He is a fellow of the Royal Aeronautical Society. Professor Tsourdos' research is focused on guidance, control and navigation for single and multiple autonomous vehicles as well asmission planning and multi-agent systems.